FOUNDATION IMPLEMENTATION PLAN

Rocks Core·오브젝트·입력 시스템 상세 구현 계획

안정성 우선 C++23 기반 라이브러리 · 객체 수명 · Windows/macOS 입력

기준일 2026.11.10 · Rocks 상세안 1.1 · 상태: 구현 계획 / 미구현·성능 미측정

Rocks 전체 구현 계획의 Core/Object Foundation과 입력 영역을 실제 작업 단위로 구체화한다. **우선순위는 안정성·수명 정확성, 확장성, 성능**이며 첫 구현은 Core 기반 라이브러리와 오브젝트 시스템이다. Windows와 macOS 개발·입력은 필수이며, 선언하지 않은 선택 장치·플랫폼·최적화는 후속으로 남길 수 있다. 지원 범위, 확장 지점, 미지원 결과, 추가 시 회귀 기준을 명확히 구분한다.

문서의 경로·타입·API·작업 ID는 신규 엔진 저장소를 위한 제안이다. 엔진 소스가 이미 구현되었거나 예시 API가 컴파일·실행 검증되었다는 의미가 아니다. 기존 전체 기능 목표와 Blueprint 제외·커스텀 언어 정책을 유지한다. 함께 제공하는 작업 목록 52개와 파일 목록 297개는 본문과 같은 데이터로 관리한다.

목차

1	안정성·확장성 원칙과 첫 완료 지점	1	14	파괴·종료·모듈 해제	22
2	모듈 의존성과 조립 위치	3	15	Core 구현 순서와 첫 실행 경로	23
3	저장소·파일 배치와 소유권	5	16	Core·Object 검증 기준	26
4	빌드 규약과 확장 API	7	17	입력 시스템 경계와 확장 계약	28
5	Base와 Platform primitive	8	18	입력 이벤트·시간·동시성	31
6	메모리와 할당기 상세	9	19	키보드·마우스·문자 입력	33
7	컨테이너·문자열·식별자·수학	11	20	게임패드·콘솔·출력 장치	34
8	Task·IO·Trace의 실행 계약	14	21	액션·컨텍스트·재바인딩	36
9	MetaSchema·Archive와 생성 코드	15	22	UI·에디터·컷씬·커스텀 언어 연결	37
10	청크 Registry와 객체 identity	17	23	재생·네트워크·헤드리스 검증	38
11	참조·소유권·쓰기 장벽 API	18	24	입력 작업 순서와 테스트 기준	39
12	생성·기본값·복제 transaction	19	25	확장 검증·위험·보류 목록	42
13	GC의 단계별 구현과 수거 규약	20	26	전체 계획과의 연결·검토 기록·참고 자료	43

1 안정성·확장성 원칙과 첫 완료 지점

1.1 변경이 국소화되는 구조

새 객체 타입, 입력 장치, 할당기, 플랫폼을 추가할 때 변경 범위는 해당 모듈과 등록 데이터로 한정한다. Core public header의 vendor enum·SDK 타입·구현 include를 늘리는 방식은 확장으로 인정하지 않는다. 다만 모든 함수를 가상 함수로 만드는 정책은 채택하지 않는다. 외부 변화가 있는 경계에는 versioned interface를 사용하고, hot loop의 leaf 구현은 concrete type·template·내부 함수로 유지한다.

설계 원칙	구현 규칙	확장성 검증
의존 방향 고정	낮은 모듈은 상위 모듈을 include/link하지 않음	dependency manifest·include/link audit
정책과 기구 분리	allocator·입력 backend·clock·serializer resolver를 주입	fake 구현으로 production 코어 실행

설계 원칙	구현 규칙	확장성 검증
등록 기반 확장	타입·장치 backend·action processor·codec은 descriptor+token	새 plugin 추가 시 Core 파일 변경 0건
수명 우선	모든 등록·callback·job·객체에 owner/generation/lease 명시	unload·취소·재등록 경합 fixture
버전 있는 데이터	TypeId/FieldId/ActionId 안정화, migration·unknown 정책	과거 fixture와 새 plugin 조합
작은 초기 범위	optional capability는 Unsupported로 반환	필수 기능 실패와 선택 기능 미지원 구분

표 1

새 확장이 공통 의미 자체를 바꾸면 ADR로 Core 변경을 허용한다. 'Core 변경 0건'은 예정한 확장 사례에 대한 기준이며 모든 미래 요구에 대한 절대 약속이 아니다. 범용 ServiceLocator·전역 singleton· `void*` 메시지 버스를 모듈 결합의 기본 수단으로 사용하지 않는다.

1.2 구현 수준의 구분

수준	이번 계획의 완료 의미	후속으로 남겨도 되는 항목
R0 · 첫 실행	Win/Mac CPU runner·메모리·최소 Array·구조화 log/CPU scope·test helpers	slab 최적화·복잡한 container
R1 · 객체 최소 수직 구현	수동 TypeInfo·Registry·Root/Weak·full GC·object/root/retire ledger와 누수 report	Clang 생성기·병렬 점진 GC·World
R2 · Foundation	CF01–CF12·Archive·Task·Core 시간·검증된 math adapter·필수 trace/analyzer/offline viewer	GC 성능 최적화·고급 plugin loader
I0 · 입력 기준 구현	synthetic 장치·event→action·재생·focus 취소	실물 장치·콘솔 SDK
I1 · 첫 실제 입력	Windows/macOS 각각 키보드·마우스·검증한 gamepad	모든 패드 모델·고급 haptics
I2 · 필수 통합 / I2+ 선택 확장	Win/Mac UI/IME·다중 사용자·게임/편집기 연결; 추가 backend·콘솔별 검증	SDK/장비가 없는 platform은 미지원 유지

표 2

R0/R1은 첫 4주의 prototype 목표이며 production Foundation 완료 선언이 아니다. Core/Object의 R2가 후속 시스템 본격 구현의 기본 선행 조건이다. 입력·렌더러·에디터·VM 때문에 Foundation 착수를 지연하지 않는다. 콘솔은 Xbox·PlayStation·Nintendo 계열 하드웨어 플랫폼을 의미하며, 개발자 명령 콘솔은 별도 UI 입력 소비자로 정의한다.

1.3 Rocks 명명·지원 기준

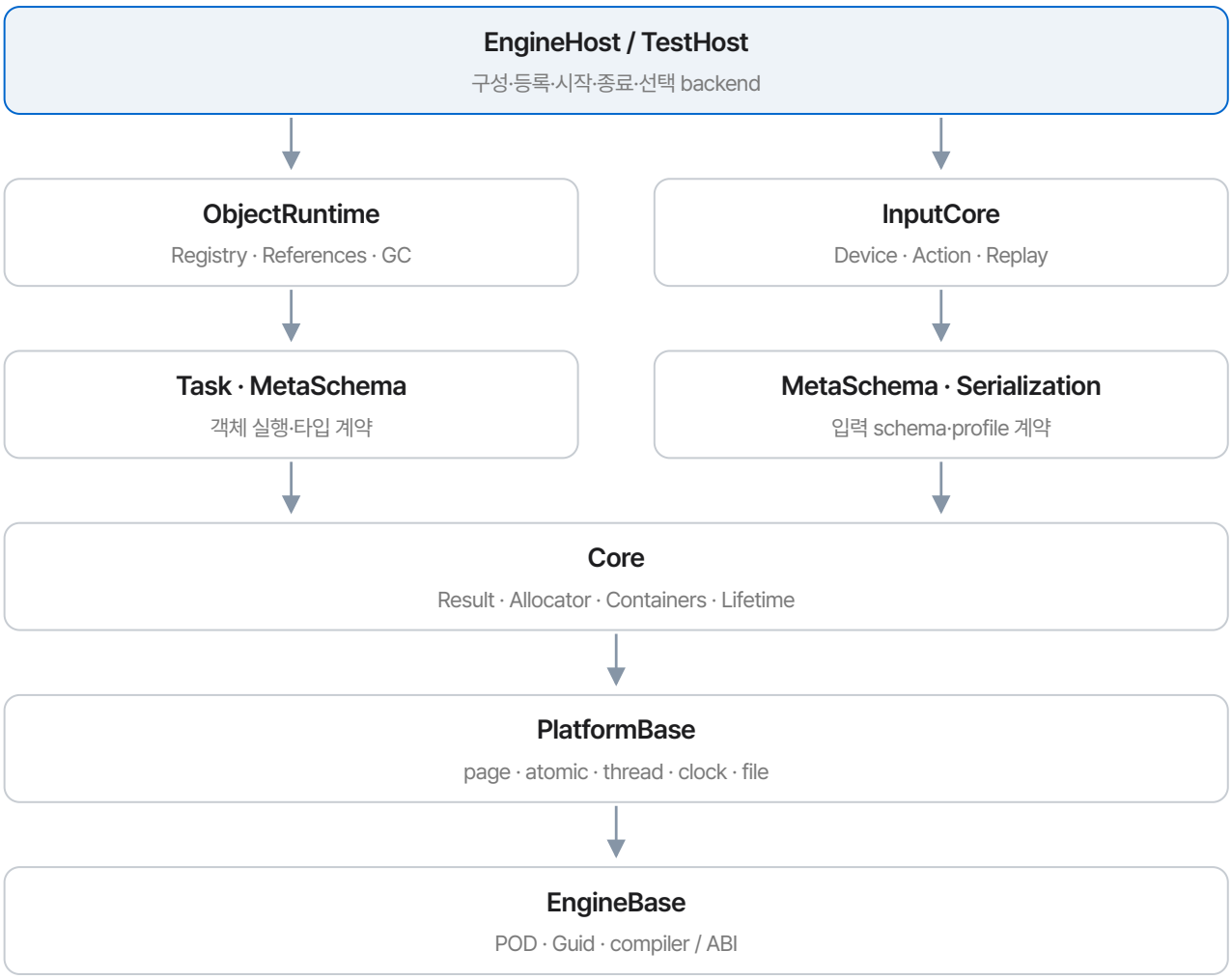
내부 C++ 타입은 namespace `rocks` 아래의 Object/Entity/Component/TransformComponent/Allocator/InputBackend처럼 역할로 명명한다. 분류용 U/F/A/I/T/E/R 접두사를 추가하지 않으며 예제에서는 namespace 내부 표기를 생략한다. 외국 SDK의 고유 명칭은 변환하지 않는다. macro에는 ROCKS_를 사용할 수 있고, foreign C ABI의 `rocks_` symbol/opaque handle 또는 소수 export+versioned function table은 실제 C/커스텀 언어 glue의 충돌·생성/해제·진단 시험 후 확정할 후보다. namespace만으로 C linkage 충돌이 해결된다고 가정하지 않는다. TypeId/FieldId는 이름 변경과 독립이다.

도구 이름은 Rocks Build·Rocks Meta·Rocks Lens이며 전체 에디터는 Rocks Studio다. 이번 계획은 구현 코드가 아닌 산출물·계약·검증 순서다. Windows x64와 macOS ARM64를 필수 개발 기준으로 두고, Intel Mac 또는 Windows ARM64는 별도 target 지원표로 검증한다. 선택 target의 부재와 두 필수 OS의 미검증을 같은 완료 상태로 처리하지 않는다.

2 모듈 의존성과 조립 위치

2.1 빌드 계층

기존 계획의 Core/Platform 경계를 상세화하여 최하위 Base header target을 추가한다. Platform이 Core의 Result/String을 사용하고 Core가 Platform을 사용하는 순환을 방지하기 위한 분리다. Base는 정수·정렬·compiler macro·POD ABI 타입만 제공하며 할당·컨테이너·OS 호출이 없다.



화살표: 기본 의존 방향 · 선택 bridge/backend는 본문에 별도 명시

그림 1 · 확장 가능한 모듈의 의존 계층 (개념도)

Target	직접 의존	공개 책임·금지 의존
EngineBase	최소 언어 지원 헤더	고정 폭 정수·ABI macro. 실행 코드 없음
PlatformBase	EngineBase·OS	page·thread·clock·file·atomic primitive. Core 금지

Target	직접 의존	공개 책임·금지 의존
Core	EngineBase·PlatformBase·선정 math private adapter	Result·allocator·container·text·math·ClockService/StepPlanner. Object/Window 금지
Task / IO / Trace	Core	각각 독립. producer는 sink만 사용; Task가 viewer/IO를 필수 링크하지 않음
TraceAnalysis / RocksLens	Core·trace format schema	capture 분석·offline timeline/report. runtime/웹 editor 비의존
Inspection	Core·주입된 snapshot provider 계약	headless 상태 조회. Object 진단은 Host가 provider로 연결
MetaSchema / Serialization	Core	schema와 binary archive. ObjectRuntime 없음
ObjectRuntime	Core·Task·MetaSchema	registry·참조·GC·생성·수명. World/Asset/VM 없음
ObjectSerialization	ObjectRuntime·Serialization	R2 객체 Archive/ID resolver bridge. ObjectRuntime 역의존 금지
PlatformWindow	Core·PlatformBase·window SDK	창·focus·text service·pointer capture
InputCore	Core·MetaSchema·Serialization	장치 상태·action 평가·profile·재생. Window/Object/SDK 없음
`InputBackend.*`	InputCore 계약·필요 PlatformWindow/SDK	backend별 선택 target. 콘솔 전용 SDK는 해당 target만
InputIntegration	InputCore·ObjectRuntime·World/UI 계약	LocalPlayer·Object·커스텀 언어 연결
EngineHost / TestHost	필요한 target들의 조합	init 순서·서비스 주입. 하위 모듈이 참조하지 않음

표 3

Core 내부 계측은 CoreTraceSink 함수 테이블을 호출한다. Trace module이 이를 등록하므로 Core→Trace 역의존이 생기지 않는다. 같은 방식으로 ObjectRuntime은 WorldEpoch를 불투명 ContextId 값으로 검사하고 World를 include하지 않는다. 모든 서비스를 하나의 공통 registry에서 임의 조회하지 않고 생성자 또는 BootContext로 필요한 계약만 전달한다.

2.2 target 구분

FoundationTests는 PlatformWindow·입력 SDK·World·GPU 없이 실행한다. InputCoreTests는 synthetic event만 사용하며 같은 CPU-only target에서 실행할 수 있다. InputDeviceProbe는 실물 장치 backend를 포함하는 별도 개발 도구다. DedicatedServer는 InputCore의 command/replay 부분만 선택 포함하고 물리 장치 backend·text service·haptics는 링크하지 않는다.

Host MetaCompiler는 LLVM/Clang과 STL을 사용할 수 있다. Target Runtime에는 generated table만 연결한다. SDK를 찾지 못한 선택 backend를 자동으로 가짜 정상 backend로 대체하지 않는다. 빌드 manifest에 Disabled/Unavailable과 사유를 기록하고 필수 target 요청이면 configuration 오류를 반환한다.

3 저장소·파일 배치와 소유권

3.1 공통 디렉터리 규약

```
Engine/
  Base/Public/Base/
  PlatformBase/{Public/Platform,Private/Win64,Private/Mac}
  Core/{Public/Core,Private}
  Task/{Public/Task,Private}
  IO/{Public/IO,Private}
  Trace/{Public/Trace,Private}
  MetaSchema/{Public/Meta,Private}
  Serialization/{Public/Serialization,Private}
  ObjectRuntime/{Public/Object,Private}
  ObjectSerialization/{Public/ObjectSerialization,Private}
  PlatformWindow/{Public/Window,Private}
  InputCore/{Public/Input,Private}
  InputBackends/{Synthetic,Windows,Mac,SDL,Steam,Console}
  InputIntegration/{Public,Private}
Tools/{MetaCompiler,InputDeviceProbe,Scaffold,RocksLens}
Tests/{Foundation,Core,Object,Input,Integration}
Build/{CMake,Policies,Manifests,Toolchains}
Schemas/{Types,Input,MigrationFixtures}
Docs/ADR/
```

public include는 `<Core/Containers/Array.h>` 처럼 모듈 namespace를 사용한다. Private 경로를 다른 target에 export하지 않는다. 공개 header는 필요한 최소 header만 include하고 독립 compile fixture를 생성한다. Engine.h 같은 전체 include는 sample에 한정한다. OS·장치 SDK header는 Private adapter에 둔다. 수학 backend는 7.4절의 검증에 따라 내부 detail inline header의 compile dependency를 허용할 수 있으나 공개 signature/ABI에 vendor 타입을 노출하지 않는다.

3.2 Core/Object의 대표 파일

경로 묶음	주요 파일	책임
Base/Public/Base	Types.h, Compiler.h, PlatformAbi.h	allocator 이전에 사용 가능한 최소 POD
PlatformBase/Public/Platform	Memory.h, Atomics.h, Thread.h, Clock.h, File.h	OS primitive의 불투명 handle
Core/Public/Core	Result.h, Error.h, Assert.h, Traits.h, Move.h	실패·언어 보조 규약
Core/Memory	Allocator.h, PageAllocator.h, Arena.h, Pool.h, AllocationTag.h	할당·회수·예산·진단
Core/Containers	Array.h, Span.h, InlineArray.h, HashMap.h, BitArray.h, SlotMap.h	GC 비의존 일반 자료구조
Core/Text·Math	String.h, StringView.h, Name.h, Guid.h, Transform.h	문자열·식별자·수학
Core/Time	ClockService.h/.cpp, StepPlanner.h/.cpp	플랫폼 독립 시간 계산 4개 파일; World 비의존

경로 묶음	주요 파일	책임
Core/Diagnostics	LogRecord.h, LogSink.h, ProfileScope.h, BootstrapLog.cpp	R0 필수 로그·최소 CPU 계측
Task·IO·Trace	TaskScheduler.h, AsyncFile.h, TraceSession.h, TraceWriter.cpp	실행·파일 비동기·필수 수집
TraceAnalysis·Inspection·RocksLens	TraceReader.h/.cpp, InspectionSnapshot.h, OfflineViewer.html/.js	R2 필수 분석·상태 조회·읽을 수 있는 timeline/report
MetaSchema·Serialization	TypeSchema.h, StableId.h, Archive.h, ObjectReferenceResolver.h	영속 스키마·객체 resolver 계약
ObjectRuntime/Public/Object	ObjectHandle.h, Object.h, ObjectRegistry.h, ObjectRefs.h	사용자가 접근하는 객체 API
ObjectRuntime/Private	RegistryChunks.cpp, RootRegistry.cpp, GcCoordinator.cpp, GcMarker.cpp	청크·root·safepoint·mark
ObjectRuntime/Private	ObjectFactory.cpp, ConstructionContext.cpp, ClassDefaults.cpp, RetireQueue.cpp	생성·기본값·정리

표 4

모듈별 Public/Private의 실제 상대 경로, 담당 작업 ID, 역할, 대표 test는 별도 파일 목록 CSV에 기재한다. 아직 작성되지 않은 제안 파일이다. 모든 파일을 첫날 생성하지 않고 작업 패키지가 해당 경로를 도입한다. 지나치게 큰 파일은 책임 단위로 분리하되 함수마다 interface 파일을 만드는 방식은 피한다.

3.3 입력 파일의 추가 위치

```
InputCore/Public/Input/
  Device.h  Control.h  Event.h  Backend.h  InputSystem.h
  Action.h  Binding.h  Context.h  PlayerInput.h
  RebindSession.h  InputProfile.h  Haptics.h  Replay.h
InputCore/Private/
  DeviceRegistry.cpp  EventQueue.cpp  TickAssembler.cpp
  ActionEvaluator.cpp  ContextRouter.cpp  ProfileCompiler.cpp
  RebindSession.cpp  HapticMixer.cpp  Replay.cpp
InputBackends/Windows/Private/
  GameInputBackend.cpp  RawInputBackend.cpp  BackendArbiter.cpp
PlatformWindow/Private/Win64/
  WindowEvents.cpp  CursorCapture.cpp  TextInputTsf.cpp
InputBackends/Mac/Private/
  AppKitInputBackend.mm  GameControllerBackend.mm  BackendArbiter.cpp
PlatformWindow/Private/Mac/
  WindowEvents.mm  CursorCapture.mm  TextInputClient.mm
InputIntegration/Private/
  ObjectActionBinding.cpp  UiInputRouter.cpp  ScriptInputBinding.cpp
```

Console 디렉터리는 adapter 계약과 빌드 선택점부터 정의한다. 공개 문서에 없는 제조사 함수명·구조체를 추정하여 API로 고정하지 않는다. 승인된 SDK를 확보한 이후 vendor private 구현과 시험 목록을 추가한다.

4 빌드 규약과 확장 API

4.1 첫 bootstrap

CMake/Ninja, C++23, Windows clang-cl 및 macOS Apple Clang을 필수 고정 manifest로 시작한다. macOS Objective-C++ .mm는 PlatformWindow/InputBackend.Mac private adapter에 한정하고 Apple framework 타입은 공개 header에 노출하지 않는다. Linux는 첫 Foundation 이후 M3의 Core CPU/headless 검증, M5 서버 통합에서 완료할 필수 후속 제품 target이다. 첫 Foundation의 필수 OS는 Windows/macOS이며 Linux를 선택 삭제하지 않는다. Runtime 예외·RTTI는 비활성화하고 warning·packing·CRT·ABI profile을 target 단위로 일치시킨다. 구체 compiler/SDK 버전은 저장소 착수 시 manifest에 고정한다. CMake option으로 Headless, Tests, Sanitizer, InputBackend를 독립 선택한다. 이후 Rocks Build가 같은 모듈 manifest를 읽도록 중간 target 모델을 유지하며 처음부터 분산 빌드 engine을 구현하지 않는다.

STL 금지는 엔진 작성 Runtime·공개 API에 적용한다. cstddef/cstdint/new 등 언어 지원 화이트리스트와 Platform private의 atomic 구현 예외를 유지한다. Host 도구와 ThirdParty 내부 사용은 허용하되 std 타입·예외·해제 책임을 엔진 경계에 노출하지 않는다. 비선택 backend를 찾기 위한 SDK 탐색조차 CoreTest configure의 필수 작업으로 만들지 않는다.

C00은 Python host의 최소 scaffold로 module/test target·manifest를 생성한다. source/SDK/Runtime VM/웹 editor를 선행 요구하지 않는다. plan 단계의 경로 이탈·충돌·필수 매개변수 검증, staging·journal·실패 rollback, 같은 출력의 timestamp 유지, 재시도 시 중복 ID/파일 방지를 TT01과 CT01로 검증한다. 사용자 파일의 변경을 발견하면 충돌로 반환한다. 전체 카탈로그·사용자 템플릿 저장·Linked 갱신은 26절의 후속 기능이다.

4.2 내부 확장과 외부 plugin ABI

같은 toolchain/build의 내부 모듈은 C++ API를 사용한다. 독립 배포 plugin은 structSize·abiMajor/Minor·FeatureId·함수 테이블·opaque handle·명시 destroy 함수의 C ABI를 사용한다. Array/String/RootRef나 native vtable을 바이너리 호환 경계로 직접 전달하지 않는다. 출력 buffer는 caller 소유 또는 provider 해제 함수와 쌍으로 제공한다.

```
struct ExtensionHeader {
    uint32 structSize;
    uint16 abiMajor, abiMinor;
    Guid featureId;
};

struct RegistrationToken {
    uint64 slot, generation;
};

Result<RegistrationToken> RegisterExtension(const ExtensionDescriptor&, ModuleLease);
Result<void> BeginUnregister(RegistrationToken);
UnregisterState PollUnregister(RegistrationToken);
```

실제 C ABI는 Base POD와 error code로 표현하며 위 Result 예시는 엔진 내부 facade다. table을 사용하는 쪽은 최소 structSize·major를 검증하고, minor 추가 field는 크기 범위 안에서만 읽는다. optional function이 null이면 해당 capability는 미지원이다. 기능 name이나 GUID만 같다는 이유로 ABI를 호환으로 판정하지 않는다.

ExtensionRegistry와 실제 DLL loader는 EngineHost 조립 계층에 둔다. Core의 ModuleLease는 loader에 의존하지 않는 코드 소유권 token이며 owner가 주입한 retain/release 계약만 갖는다. ObjectRuntime/InputCore는 이 token을 보존할 뿐 Host를 호출하지 않는다. 초기 static module은 shutdown까지 유지되는 lease로 구현하고 C27에서 dynamic loader를 추가한다.

등록은 Validate → Stage → Commit의 원자적인 transaction이다. 실패 시 이미 등록한 항목을 역순 해제한다. 해제는 신규 요청 차단 → subscription/task 취소 → callback drain → instance/metadata/backend lease 종료 → table 제거 순서다. 남은 사용자가 있으면 Busy를 반환하고 실제 DLL unload를 보류한다. native layout 갱신은 기본 재시작 경로를 사용한다.

4.3 코드 서식·clang-format 표준

사람이 알아보기 좋은 코드 스타일 유지를 코드 작성과 리뷰의 명시적 원칙으로 적용한다. 읽기 쉬운 이름·적절한 빈 줄·짧고 명확한 함수 구조를 요구하며 formatter 통과만으로 가독성 검토를 대체하지 않는다.

엔진 소유 코드는 **탭 문자를 사용하지 않고 스페이스 4칸으로 들여쓴다.** 함수·타입 정의 사이, 검증·상태 갱신·결과 반환 등 의미가 다른 처리 단계 사이에는 빈 줄을 둔다. 관련된 짧은 선언은 묶되 여러 문장을 한 줄에 압축하지 않는다. 긴 함수·복잡한 조건·여러 책임을 서식만으로 해결하지 않고 작은 함수와 명확한 이름으로 분리한다.

`clang-format` 버전과 `.clang-format`을 저장소에 고정한다. 에디터 저장 시 포맷·선택 영역 포맷·로컬 검사·CI가 같은 실행 파일과 설정을 사용한다. `.editorconfig`는 `indent_style = space`, `indent_size = 4`, `tab_width = 4`, LF·파일 끝 개행·불필요한 행 끝 공백 제거를 명시한다. 열 제한 100자는 초기 기본값이며 줄바꿈 시 가독성을 우선한다. C++/Objective-C++는 같은 규칙을 적용한다.

첫 C00 산출물에 `.editorconfig`, `.clang-format`, `Docs/CodingStyle.md`를 포함한다. CT01에는 설정 파싱, 실제 tab 부재, 같은 입력의 포맷 재실행 동일성, 의도적으로 서식을 깨뜨린 fixture의 실패를 추가한다. PR에서는 `clang-format --dry-run --Werror`에 해당하는 검사를 수행하고 generated 파일은 생성 단계부터 같은 설정으로 출력한다. formatter 버전 변경은 별도 변경과 diff 검토로 처리한다. 외부 vendor 원본은 기본적으로 재포맷하지 않고 엔진 소유 adapter에 Rocks 규칙을 적용한다.

`SeparateDefinitionBlocks: Always`와 짧은 함수/블록/조건/반복문의 한 줄 압축 금지 설정으로 정의 사이의 간격을 유지한다. 자동 도구가 처리 단계의 의미를 판단하지는 못하므로 함수 내부의 논리적 빈 줄·책임 분리는 리뷰 항목으로 남긴다. 문서의 C++ API 예제에도 같은 formatter를 적용하되, 서식 통과를 API 컴파일·엔진 구현 테스트의 통과로 해석하지 않는다. clang-format 공식 설정

5 Base와 Platform primitive

5.1 allocation 이전 계약

Base는 uint8/32/64, Guid POD, checked integer helper의 선언, compiler annotation, opaque handle, ByteView, OsStatus만 갖는다. OsStatus는 OS 오류 숫자와 범주를 갖는 POD이며 String이나 heap allocation을 포함하지 않는다. Core가 이를 Error/Result로 변환한다. 최하위 fatal 경로는 고정 stack buffer와 OS stderr/debug 출력만 사용한다.

```
struct PageRange {
    void* address;
    uint64 reservedBytes;
};

OsStatus ReservePages(uint64 bytes, PageRange* out);
OsStatus CommitPages(PageRange, uint64 offset, uint64 bytes);
void ReleasePages(PageRange);
uint64 MonotonicTicks() noexcept;
uint64 MonotonicFrequency() noexcept;
OsStatus StartThread(ThreadEntry, void* context, ThreadHandle* out);
```

Reserve/Commit/Decommit의 정렬·범위·overflow·0-byte 의미를 명시한다. page guard와 allocation failure injector는 test implementation으로 제공한다. Monotonic clock과 UTC wall time은 별도 타입이며 timeout은 monotonic만 사용한다. timer 해상도와 scheduling precision을 동일한 값으로 보고하지 않는다.

Windows/Mac의 page·clock·atomic·sync는 C02부터, thread·join은 C16, offset 파일 구현은 C18에서 두 target을 함께 검증한다. path encoding·case sensitivity·Unicode 파일명·부분 read·rename 실패·열린 파일 교체·시간 해상도의 차이를 contract fixture로 기록하고 동일 OS 동작을 가정하지 않는다. Mac file/thread/clock은 Foundation Gate의 필수 파일·시험 목록이며 SDL 선택 adapter에 포함하여 연기하지 않는다.

5.2 Atomic·동기화·파일

Atomic32/64/Pointer는 memory order enum과 Load/Store/Exchange/CompareExchange/FetchAdd를 제공한다. public에는 std::atomic layout이 없다. 최초 구현은 Platform private에서 검증된 std::atomic을 불투명 정렬 저장소에 생성하며 size/alignment를 static_assert한다. opaque ABI 크기를 만족하지 않는 target은 build 오류 또는 다른 backend 구현을 선택한다. std 라이브러리 허용 범위를 몰래 확대하지 않는다.

허용하지 않는 load/store/failure memory order는 compile/debug 진단한다. relaxed counter는 수명 publication에 사용하지 않는다. 초기 mutex/event/semaphore 기반 구현을 기준으로 유지하며 lock-free 여부는 capability로 조회한다. C++ 메모리 모델을 우회하는 volatile 기반 동기화는 금지한다. 파일은 명시 offset·부분 read/write·취소·native error를 반환하고 handle 재사용에는 generation을 사용한다.

테스트는 page 정렬·commit 실패·멀티스레드 publication·mutex 재진입 정책·event 신호 유실·thread join·부분 IO·clock 역행 모의를 포함한다. Platform 내부에서 Core allocator를 호출하는 순환이 없는지 symbol/include audit로 확인한다.

6 메모리와 할당기 상세

6.1 공통 API와 실패 의미

```
struct AllocationRequest {
    uint64 bytes;
    uint32 alignment;
    MemoryTag tag;
};

struct Allocator {
    virtual Result<void*> TryAllocate(AllocationRequest) noexcept = 0;
    virtual void Free(void*, uint64 bytes, uint32 alignment) noexcept = 0;
    virtual ~Allocator() = default;
};

Result<uint64> CheckedAllocationSize(uint64 count, uint64 elementSize);
```

0-byte는 성공한 null로 통일하고 Free(null)는 무효 연산이다. alignment는 지원 범위의 2의 거듭제곱이어야 한다. bytes×count와 정렬 올림 모두 overflow를 검사한다. allocator는 그 storage를 사용하는 모든 owner보다 오래 유지해야 한다. Free의 크기·정렬·allocator identity 오류는 debug 진단하며 ABI 경계에서 할당과 해제 주체를 쌍으로 전달한다.

초기 순서는 OS page → 단순 heap adapter → LinearArena/Pool → 계측 → slab/thread cache다. 처음부터 빠른 general allocator를 완성하려고 R1을 지연하지 않는다. Object 전용 nonmoving pool은 Registry slot 저장소와 독립이다. 큰 객체·큰 정렬은 별도 page fallback을 사용한다.

6.2 구현 정책

구현	핵심 상태	먼저 검증할 실패
PageAllocator	reserve/commit 구간·page 크기	범위 초과·부분 commit 실패
LinearArena	block·cursor·generation	Reset 뒤 view 사용·overflow
StackArena	marker stack	역순 위반·소멸자 누락
FixedPool	slot bitmap·generation	double free·다른 pool free
SlabAllocator	size class·owner shard·remote free	thread 종료 후 remote free 잔존
BudgetAllocator	used/reserved/peak·상한	상한 초과의 명시 실패·잘못된 rollback
TrackingAllocator	tag·sample·원본 allocator	tracking 재귀 할당·진단 중 deadlock

표 5

arena Reset은 관련 task/read scope의 종료 이후에만 허용한다. 향후 GPU upload arena는 별도 fence 계약을 추가하며 CPU arena가 이를 암묵 처리하지 않는다. cross-thread free는 owner shard의 bounded queue 또는 잠금 기준 경로를 사용하고 overflow의 fallback lock 경로를 명시한다. owner thread 종료 시 queue를 drain하고 남은 slab의 소유권을 중앙 pool로 이관한다.

할당 추적은 미리 확보한 buffer를 사용한다. tracking 실패가 실제 allocation 성공을 손상시키지 않도록 loss counter를 기록한다. recoverable budget 실패는 Result, 회복 불가능한 전역 OOM은 최소 진단 후 종료로 구분한다. 실패 주입은 N번째 allocation 실패와 특정 tag 실패를 지원한다.

손실을 허용하는 상세 allocation event와 누수 판정에 사용하는 owner/count ledger를 구분한다. ledger는 allocation publication 전에 필요한 bookkeeping 용량을 확보하거나 명시 admission 실패를 반환한다. 계측 이벤트가 누락되면 그 구간을 불완전으로 표시하고, 독립적으로 보존한 ledger·scope baseline 근거 없이 누수 0을 선언하지 않는다. crash 시의 일부 기록 회수와 정상 종료의 완전한 closure를 같은 결과로 집계하지 않는다.

6.3 안정성·누수 ledger와 완료 조건

메모리 누수 0을 정상 종료 Gate로 두며 allocation·managed graph·task/callback·IO buffer·type/module·trace를 별도 owner scope와 generation으로 계상한다. R1은 object/root/retire 수, native heap과 graph의 retained 원인, 실패 단계의 잔존을 필수 출력한다. full GC만으로 native/backend 자원이 회수되었다고 판단하지 않는다. scope 종료는 owner 0과 명시한 공유 cache baseline 복귀로 검사하며 정상 shutdown은 엔진이 추적하는 소유 자원 잔존 0을 요구한다.

RootRef를 의도적으로 유지한 DestroyRequested 객체는 보존 사유를 출력한다. root 제거·backend 완료 후에는 회수를 검증한다. cache/pin allowlist는 owner·상한·만료/해제 조건을 갖고 증가하는 누수를 cache로 재분류하지 않는다. timeout/cancel은 외부 완료 증명이 아니다. 미완료 자원은 bounded retained/quarantine으로 진단하고 신규 admission을 제한하며 안전한 backend 종료 경로가 끝나기 전에 강제 free하지 않는다. 회수 불완전은 실패다.

N-fail·부분 생성 rollback·취소·늦은 완료·중복 callback·재진입·module 종료와 1,000회 boot/shutdown을 CT03/15/20–22·NL01–03/06–08로 검증한다. 누수·UAF·double free·data race는 성능 수치로 상쇄하지 않는다. 모든 환경의 무결함을 유한한 시험으로 입증하였다고 주장하지 않고 SDK/driver와 자체 ledger의 관찰 범위, 미실행 sanitizer를 함께 보고한다.

7 컨테이너·문자열·식별자·수학

7.1 container의 공통 수명

```
template <class T>
class Array {
public:
    explicit Array(Allocator&) noexcept;
    Array(Array&&) noexcept;
    Result<void> TryReserve(uint64 capacity);
    Result<void> TryPushBack(const T& value);

    template <class... A>
    Result<T*> TryEmplaceBack(A&&...);
    Result<void> TryAppendRange(Span<const T> values);
    Result<Array> TryClone(Allocator&) const;
    Span<T> View() noexcept;
    void Clear() noexcept;
};
```

동적 owning container는 기본 move-only로 시작하고 실패 가능한 복제는 TryClone으로 명시한다. element의 생성·복사·이동·소멸 중 해당 연산에 사용하는 경로는 noexcept여야 한다. 업무 초기화 실패는 constructor 밖의 Initialize로 분리한다. TryReserve 실패는 기존 데이터·capacity·iterator를 보존하고 성공한 재할당은 모든 기존 view를 무효화한다.

TryPushBack과 TryAppendRange는 자기 buffer alias를 안정화한 후 재할당한다. 임의 variadic TryEmplaceBack 인자가 자기 buffer를 참조하면서 grow가 필요한 경우는 최초 계약에서 금지하고 진단 가능한 alias를 검사한다. 이 경우 caller는 별도 임시 값 또는 alias-safe API를 사용한다. shrink/erase의 이동과 소멸 횟수도 문서화한다.

portable byte relocation은 trivially-copyable complete element와 storage/lifetime 조건을 만족할 때만 사용한다. implicit-lifetime이라는 성질만으로 임의 이동을 승인하지 않는다. nontrivial 타입은 placement move+destruct를 사용한다. compiler extension은 capability·버전별 별도 최적화다. D1

7.2 자료구조별 첫 범위

타입	초기 선택	확장 지점·검증
Span/StringView	pointer+count 비소유	경계·빈 view·비동기 보관 금지
Array/InlineArray	연속 buffer·기하 성장	allocator 교체·alias·소멸 계수
HashMap/HashSet	open addressing·control bytes	collision hash·rehash rollback·erase
BitArray	word buffer	마지막 word mask·atomic 버전은 별도
SlotMap/ChunkedArray	stable chunk·generation	재사용·wrap 격리·iterator epoch
Function/Delegate	move-only owning callable·SBO	move/destroy function table·해제 token
UniquePtr/SharedPtr	allocator/deleter 명시	cycle 금지·copy refcount overflow

표 6

HashMap 순회 순서는 영속 데이터·결정적 게임 로직의 기준이 아니다. canonical serialization은 key를 별도 정렬한다. nontrivial key/value의 실패 가능한 준비는 bucket 공개 전에 완료한다. 컨테이너 조작에 lock이 자동 포함된다고 가정하지 않는다. GcArray는 11절의 owner-aware edge 규약을 가진 별도 타입이다.

7.3 text·Name·Guid·Transform

String은 UTF-8 소유, StringView는 길이 포함 view다. byte count와 code point/grapheme count를 같은 API로 노출하지 않는다. Core에는 검증·변환의 최소 기능을 두고 정규화·BiDi·shaping은 상위 text adapter로 확장한다. Name은 세션 내 intern ID이며 영속 저장은 문자열 또는 stable GUID다. NamePool은 명시 init/shutdown·shard lock·immutable 문자열 storage를 사용한다.

Guid의 128-bit POD 표현은 C01의 Base/Guid.h에 둔다. C13의 Core/Guid.h는 생성·parse·format helper만 추가한다. Guid 생성은 OS random backend를 사용하고 TypeId/FieldId/ActionId는 source control의 manifest로 유지한다. 타입 rename으로 ID가 바뀌지 않는다. hash는 runtime lookup용과 콘텐츠 digest용을 구분한다. Path는 플랫폼 정규화·case 정책을 노출하며 문자열의 locale 변환으로 파일 identity를 결정하지 않는다.

LocalTransform은 double translation·quaternion·diagonal scale, WorldAffine은 affine matrix다. 비균일 scale과 rotation이 만드는 shear를 계산 결과에 보존한다. KeepWorld reparent가 local TRS로 표현되지 않으면 명시 오류를 반환한다. normal inverse-transpose·음수 determinant·singular inverse·bounds를 CF04/CT06에 포함한다. 산술 알고리즘은 검증된 backend를 adapter로 호출하며 자체 scalar/SIMD 수학 라이브러리를 새로 구현하지 않는다. 파일/ABI layout은 backend SIMD layout과 분리한다.

7.4 수학 backend 선정·adapter Gate

C15는 수학 알고리즘 개발 과제가 아니다. GLM·Realtime Math·DirectXMath 등 검증 가능한 라이브러리를 후보로 비교하고, 필요한 double translation/affine·quaternion·inverse·decomposition·bounds 연산과 x86/ARM64 SIMD 경로를 확인한 뒤 선정한다. DirectXMath를 품질 문제로 배제하거나 확정 backend로 지정하지 않는다. 공개 API의 명명·코드 스타일과 backend의 정확성·성능은 별도 기준으로 평가한다. 필요한 연산이 후보에 없으면 다른 검증된 라이브러리의 adapter 또는 명시 Unsupported를 검토하며 자체 ScalarMath.cpp로 보충하지 않는다. D17, D18, D19

선정표에는 license·유지보수/고정 revision·STL/예외/RTTI 경계·precision·alignment·CPU dispatch·NaN/singular 오류·컴파일 비용과 코드 스타일/adapter 유지보수성을 기록한다. 공개 API는 `rocks::Vec3`, `rocks::Quat`, `rocks::Mat4` 처럼 Rocks 표기를 사용하고 RTM의 함수 스타일을 포함한 vendor type·macro·ABI는 private adapter 내부에만 둔다. adapter는 좌표/단위/실패·저장 형식만 변환하며 vendor type을 public header·Archive·C ABI에 노출하지 않는다. x86 기준과 ARM64 결과를 수동 golden/별도 고정밀 계산 및 backend가 제공하는 portable/reference 경로와 비교한다. 같은 구현을 복사한 oracle은 사용하지 않는다. 정확성·성능은 고정 workload의 bench/conformance Gate로 판정한다. fast-math·FMA·denormal 차이는 허용 오차와 재현 범위에 기록하고 bitwise 동일성을 임의 보증하지 않는다.

후보 manifest·ADR·MathBackendAdapter/TransformAdapter·conformance 시험이 C15 산출물이다. 라이브러리를 선정하기 전 상태는 미결정이며 candidate 이름만으로 지원 PASS를 선언하지 않는다. 선정 실패는 C15 미완료이고 Object R1을 임시 수학 구현 때문에 지연하지 않는다.

Vec3/Quat/Mat4는 f32 local/render 저장형이며 f64 Vec3d/Mat4d/WorldPosition을 별도 제공한다. vendor register·padding·CPU별 ABI를 serialization·VM·C ABI로 전달하지 않는다. DirectXMath의 f32 범위와 f64 World 요구를 분리하여 검증한다. column vector `p'=M×p`, `world=parent×local` 규약을 유지하고 row-vector backend의 순서·전치 변환을 adapter에서 처리한다. 전역 원점 차감은 f64에서 수행한 뒤 범위를 확인하여 f32로 변환한다.

공개 signature와 storage ABI에 vendor 타입을 노출하지 않되, vendor header 완전 비공개와 모든 작은 연산의 inline·무비용 wrapper를 동시에 보장하지 않는다. 내부 detail inline header의 제한된 compile dependency와 private translation unit+LTO/batch 방식을 비교한다. 전자는 macro 격리·재컴파일 범위, 후자는 호출·load/store·shuffle 비용을 측정한다. 단일 벡터 연산마다 virtual dispatch/CPU 검사를 수행하지 않으며 batch kernel 진입에서만 runtime ISA를 선택한다. baseline x64/SSE2와 ARM64/NEON을 검증하고 선택 AVX2/FMA는 CPU·OS state 조건 및 baseline/확장 코드 분리를 확인한다.

MX01–08은 API/의존·수치 예외·규약 왕복·큰 좌표·ISA/플랫폼·batch/ABI·성능/compile 비용·업데이트 재현을 검사한다. 공개 math 정책의 같은 ID를 재사용하며 CT06에 연결한다. 지원하지 않는 CPU capability를 mask한 실행, unaligned/tail/alias 경계, near-singular matrix, NaN/Inf·signed zero·subnormal을 포함한다. 일반/estimate 오차와 network 결정성 subset을 구분하고 전체 Core에 fast-math를 적용하지 않는다. 기준 workload·compiler/flags·revision을 기록한 실제 benchmark 전에는 후보 성능 우위를 선언하지 않는다.

7.5 Core 시간·가변/고정 step

Core/Time의 ClockService.h/.cpp와 StepPlanner.h/.cpp를 정확히 4개 계획 파일로 추가한다. Platform clock을 주입하고 Monotonic/Frame/Game/Fixed/UI clock을 명시 ID·epoch로 구분한다. monotonic timeout·IO·trace·UI pump는 game pause/dilation과 독립이다. World·PIE·physics 연결은 상위 모듈이며 Core가 해당 header를 include하지 않는다.

가변 Update는 frame당 1회 delta를, 고정 Update는 scaled accumulator에 realDelta×scale을 더한 뒤 고정 h를 소비한 StepPlan을 받는다. time dilation은 h를 매번 변경하지 않고 step 실행 빈도에 적용한다. scale은 finite·0 이상, 0은 pause다. 음수는 역재생이 아닌 오류이며 seek/replay는 별도 경로다. scale을 delta와 accumulator에 중복 적용하지 않는다. fixed/variable 양쪽 등록은 명시 선택이고 같은 동작의 중복 실행을 방지한다.

ClockService의 owner lane이 step을 commit하며 worker에는 immutable TickContext만 전달한다. pause 중 고정 single-step은 정확히 h 1회, variable debug-step은 명시 delta 1회다. max frame delta/substeps/debt와 overflow 실패를 선언한다. interactive debt 폐기는 버린 시간을 report하고 authoritative server는 처리하지 않은 tick을 처리된 것으로 표시하지 않는다. worker 결과는 tick/context epoch를 확인하여 commit하며 resume 시 오래된 입력 edge를 재발사하지 않는다.

ID	시간·병렬 실행의 필수 시험
TK01	30/60/144 FPS·가변 delta 합·고정 tick 수/변위·보간 golden
TK02	scale 0/0.1/1/2·변경 경계·pause/resume·fixed exact-one single-step·NaN/음수 거부
TK03	hitch·불연속·상한/overflow·debt 정책·server 미처리 tick 보고
TK04	unscaled UI/IO/timeout·pause 중 입력 pump·old edge 중복 0
TK05	0/1/N worker·순서 교란·취소·context 교체·GC 경계의 의미 동일·stale commit 0
TK06	read/write conflict·DAG cycle·reader 종료·command commit·shutdown 경합

표 7

8 Task·IO·Trace의 실행 계약

8.1 Task API

```
struct TaskDesc {
    TaskLane lane;
    Priority priority;
    CancelToken cancel;
    FailurePolicy dependencyFailure;
    Function<void(TaskContext&)> body;
};

Result<TaskHandle> Submit(TaskDesc&&, Span<const TaskHandle>);
Result<TaskHandle> Then(TaskHandle, TaskDesc&&);
void RequestCancel(CancelSource&) noexcept;
```

Submit 성공 시 capture·prerequisite 목록을 task-owned storage로 이관한다. 요청자의 Span을 보관하지 않는다. capacity·shutdown·잘못된 prerequisite는 명시 오류이며 실패한 제출에는 handle이 없다. node는 queue·실행·결과 observer의 모든 참조가 종료된 뒤 재사용한다. completion release publication과 observer acquire를 연결한다.

실행 payload와 완료 관찰 record의 수명을 분리한다. body 반환 또는 queued 실행권 취소가 선형화되고 실행 lease가 0이 된 뒤, callable/capture와 불필요한 prerequisite hold를 지정 cleanup lane에서 정확히 1회 해제한다. callable destructor 반환까지 code ModuleLease를 유지하며, 그 cleanup 완료 후에만 Succeeded/Failed/Cancelled를 release 게시한다. 취소 요청 자체는 terminal이 아니다. TaskHandle이 보존하는 완료 record에는 상태·소유 진단 결과만 남고 실행 capture는 남지 않는다. queue tombstone과 observer가 남은 node의 재사용은 계속 금지한다. IO/GPU/독립 callback으로 이관한 lease는 각 owner가 별도로 drain한다. 현재 void task의 terminal에 명시적으로 이관되지 않은 숨은 RootRef를 보존하지 않는다. RunOnTerminal continuation은 자신의 별도 payload·수명으로 실행하며, 종료 시 terminal publication에 필요한 cleanup lane을 먼저 중지하지 않는다.

Submit admission은 callable을 이동하기 전에 실행 node의 내장 cleanup link/state 또는 해당 lane의 cleanup slot까지 확보한다. 실패 시 caller의 TaskDesc/capture 소유권을 유지하며 유효 handle을 반환하지 않는다. 승인 후 cleanup 전달은 기존 node를 intrusive teardown queue로 이관하여 추가 할당·일반 Submit·별도 task node·사용자 queue 여유를 요구하지 않는다. 따라서 포화나 일반 admission 종료로 cleanup enqueue가 실패하지 않는다. capture destructor는 scheduler/queue/registry gate를 모두 해제한 뒤 지정 lane에서 실행한다. destructor는 noexcept·비차단 계약을 따르며 비동기 후처리가 필요한 자원은 미리 backend lease로 이관한다. shutdown은 이 독립 cleanup queue와 실행 중 destructor까지 drain한 후 lane을 종료한다. CT08은 일반 node pool/queue를 포화시키고 외부 TaskHandle을 유지한 상태에서도 cleanup과 terminal이 진행되는지 확인한다.

기본 dependency failure는 body 미실행과 terminal 전파다. RunOnTerminal 정리 continuation만 실패·취소도 처리한다. cancellation은 side effect rollback이 아니며 승인된 task는 Succeeded/Failed/Cancelled 중 1개를 정확히 1회 publish한다. Game/Render의 상호 wait와 lock 보유 wait를 금지한다. 초기에는 mutex queue와 명시 worker pool, 이후 동일 테스트를 통과한 work stealing을 선택한다.

8.2 GC-aware 실행과 종료

Object mutator는 첫 구현에서 Game lane 1개다. 일반 worker는 immutable/POD를 처리한다. 등록된 짧은 object reader는 read phase에만 접근하고 safepoint 전에 Borrow를 해제한다. 정지 요청을 받은 worker가 CPU slot을 점유한 채 대기하여 GC worker를 고갈시키지 않도록 작업 반환 또는 GC 참여 방식을 사용한다.

scheduler shutdown은 일반 제출 차단 → owner 취소 → 제한된 teardown capability 제출 → terminal/callback drain → worker join 순서다. capability는 무제한 신규 게임 task 권한이 아니다. 필요 resource owner를 scheduler보다 먼저 파괴하지 않는다. timeout은 원인과 미완료 작업을 보고하며 유효한 참조를 강제 free하지 않는다.

8.3 IO·Trace의 확장

AsyncFile은 ReadAt/WriteAt, CancelTicket, terminal callback lane을 제공한다. OS backend 차이는 Private에서 처리하고 initial fallback worker IO는 별도 IO pool에 한정한다. Core task queue를 장시간 동기 파일 대기로 점유하지 않는다. 취소 후에도 OS completion은 도착할 수 있으므로 request generation·buffer lease를 terminal drain까지 유지한다.

R0/C06부터 구조화 log·고정 buffer sink·최소 CPU scope·실행 요약이 필수다. R1에서는 object/root/GC phase/retire/heap ledger를 동일 session으로 연결한다. R2/C18은 CPU/Task/GC/memory/IO trace 수집, headless inspection/capture/report, analyzer와 기본 offline timeline viewer를 제공한다. C26은 필터·thread timeline·Task dependency·GC 구간·memory graph·root chain·p50/p95/p99를 읽을 수 있는 결과를 필수 검증한다. binary 파일 생성만으로 완료하지 않는다. C30은 sampling·압축·대형 trace 탐색·overhead의 미세 최적화만 선택이다.

producer는 사전 확보 bounded ring에 숫자 event를 기록하고 포화 시 실행을 대기시키지 않는다. loss 수·구간·high-water를 기록하며 viewer에 불완전 span을 표시한다. allocator 계측은 추적되지 않는 예약 storage·재귀 guard를 사용하고 allocator lock 안에서 formatting/일반 allocation/file IO를 금지한다. snapshot은 provider의 safepoint/read 계약으로 복사하고 실행 중 객체 graph를 무잠금 순회하지 않는다.

문자열·타입명·event schema는 session-owned metadata로 보존한다. module 주소를 unload 이후 역참조하지 않으며 sink 해제는 admission 차단→in-flight drain→ModuleLease 해제다. 정상 종료는 최종 GC/runtime producer 종료→trace drain/flush→writer join→allocator 종료다. writer는 일반 scheduler가 종료한 뒤에도 drain할 독립 경로를 제공한다. crash/hang은 사전 확보 breadcrumb·회수 가능한 chunk의 제한 시간 best effort이며 heap/registry 재순회·무제한 flush를 금지하고 truncated/incomplete 상태를 출력한다.

8.4 안정성 우선 병렬 실행

Task/TickGraph는 phase·prerequisite·read/write set·owner context를 선언한다. 초기 Object field mutator는 Game lane 1개이며 native SoA·animation pose·culling·decode·독립 query·packet 준비를 worker로 분배한다. worker 결과는 command/snapshot으로 반환하고 owner가 generation/tick을 확인한 commit phase에서 적용한다. physics 내부 병렬화와 외부 mutation 권한을 혼동하지 않는다.

독립 작업에 병렬 실행을 적용하되 0/1/N worker 기준을 유지한다. 결정적 결과가 필요한 merge는 완료 순서가 아닌 stable key, random stream은 tick/entity로 정한다. lock-free·work stealing·parallel GC는 소유권·진척·race/fault 검증 이후 선택한다. backend API의 thread 제약은 지키되 그 제약으로 모든 scene 준비를 단일 thread로 제한하지 않는다.

9 MetaSchema·Archive와 생성 코드

9.1 schema와 runtime ops의 분리

MetaSchema의 TypeSchema는 TypeId·schemaVersion·FieldId·wire type·flags·상속 ID를 보관한다. Object 포인터나 constructor가 없다. ObjectRuntime의 RuntimeTypeOps가 construct/destroy/trace/accessor와 TypeSchema를 연결한다. 상위 모듈이 schema를 조회하는 데 Object 생성이 필요하지 않아야 한다.

```
struct RuntimeTypeOps {
    TypeId type;
    uint32 size, alignment;
    ConstructObjectFn construct;
    DestroyObjectFn destroy;
    TraceObjectFn trace;
    ModuleLease module;
};

Result<TypeRegistrationToken> RegisterObjectType(const TypeSchema&, RuntimeTypeOps);
Result<void> ValidateTypeGraph(TypeId);
```

RegisterObjectType은 TypeSchema와 중첩 field/name/attribute 배열·문자열·속성 payload를 검증하여 registration 소유의 불변 snapshot으로 깊은 복사한다. 전달한 const 참조·Span·scratch arena를 함수 반환 후 보관하지 않는다. R1 C07은 고정 Guid/POD 배열과 Core의 소유 byte buffer로 필요한 subset을 복사하여 C13 String/Name 도입을 기다리지 않는다. 검증·복사 실패는 등록 전부를 rollback하고 완성된 revision만 원자 게시한다. generated static table도 첫 구현은 같은 복사 경로를 사용한다. 코드 ModuleLease는 metadata 저장소 소유권의 대체물이 아니다. Object·visitor·Archive reader·비동기 schema 소비자는 해당 registration revision lease를 유지하고, 마지막 소비자와 destructor 종료 전 그 revision의 ops/code/data를 회수하지 않는다. 초기 구현은 active TypeId의 비호환 재등록과 제자리 table 교체를 거부하며 unregister는 소비자가 남으면 Busy를 반환한다. native layout 변경은 process restart 기본값을 유지한다.

최초 R1은 수동 등록 테이블로 완성한다. 최소 contract가 고정된 뒤 Clang MetaCompiler가 동일 형식의 table과 accessor를 생성한다. 수동·generated visitor가 같은 graph를 열거하는지 대조한다. non-standard-layout 클래스에는 offsetof를 일반 적용하지 않고 typed accessor를 생성한다.

9.2 Archive 계약

Archive는 FieldId·wire type·length payload를 사용하고 byte order·길이·깊이·압축 해제 상한을 검사한다. reader/writer에는 ByteReader/ByteWriter와 ObjectReferenceResolver를 전달한다. ObjectHandle은 영속 저장하지 않는다. Foundation fixture의 local ObjectId와 후속 package ExportId/AssetId·network ID는 resolver adapter가 변환한다.

unknown field는 Runtime read에서 건너뛰며, editor round-trip 보존이 필요한 유형은 UnknownFieldBag에 원 payload를 제한된 크기로 유지한다. unknown TypeId의 인스턴스 생성은 실패하거나 명시 opaque asset record로 보관한다. 크기·소멸 규칙을 모르는 Object를 임의 생성하지 않는다. migration은 old schema→new schema의 검증된 함수이며 삭제 ID는 tombstone으로 예약한다.

9.3 Clang 연결 순서

compile database/target define/include와 annotation을 입력으로 AST를 수집한다. MetaParse용 generated stub으로 첫 include 순환을 해소하고 내용이 바뀐 출력만 원자 교체한다. 생성기·schema·include closure·target ABI가 cache key에 포함된다. generated C++에 formatter를 적용하는 action은 formatter executable/version digest, .clang-format과 실제 읽은 style 설정의 digest를 명시 입력으로 갖는다. 생성→format을 하나의 action 내부 단계로 실행하거나 raw와 formatted 산출물을 구분한 두 action으로 실행하며, 최종 경로의 producer는 하나다. formatter 변경 시 그 action과 후속 compile key가 무효화된다. 최종 바이트가 동일하면 출력 timestamp를 보존한다. CT12는 config만 변경·formatter binary만 변경·동일 출력의 세 분기와 원격 stale artifact 거부를 검증한다. Host 도구의 LLVM/STL은 Runtime target에서 제거한다. LibTooling과 compile database는 공식 문서의 인터페이스를 따른다. D2, D3

초기 지원은 단일 Object 주상속, 지원된 primitive/value/container/reference field다. 임의 template reflection·union·bitfield·복잡한 다중 상속은 UnsupportedMetadata로 명확히 진단한다. 필수 샘플이 필요로 하는 subset부터 구현하고 후속 타입을 등록 규약으로 확장한다.

10 청크 Registry와 객체 identity

10.1 저장 형식

ObjectHandle은 index32+generation32, index 0은 null이다. 기본 청크는 64K slot이며 디렉터리는 주소가 고정된 atomic pointer 배열로 예약한다. slot size·chunk 크기는 내부 상수이고 파일 포맷에 기록하지 않는다. 32-byte slot 가정의 약 2 MiB/청크 수치는 예산 예시이며 static_assert와 실제 측정으로 확정한다.

```
struct ObjectHandle {
    uint32 index, generation;
};
enum class ObjectState : uint8 {
    Free,
    Reserved,
    Constructing,
    Published,
    DestroyRequested,
    Retiring
};

struct ObjectSlot {
    AtomicPointer address;
    Atomic32 generation;
    Atomic32 state;
};
```

위 slot 코드는 publication 필드만 표시한 개념 예시다. address·generation32·state32를 분리하고, 나머지 TypeId/epoch는 GC gate와 객체 상태로 보호한다. 성능 최적화를 이유로 32-bit generation 범위를 암묵 축소하지 않는다.

디렉터리 release publication 이후 reader acquire가 완성된 청크를 관찰한다. slot의 상태·주소·세대를 서로 다른 atomic에서 읽었다는 이유만으로 일관 snapshot이 자동 보장되지 않는다. 최초 구현은 Registry/GC gate로 조회·publication·회수를 직렬화하고, lock-free Resolve는 별도 증명과 stress 이후 도입한다.

10.2 API와 조회 조건

```
class ObjectRegistry {
public:
    template <class T>
    Result<RootRef<T>> NewObject(const ObjectInit&);

    template <class T>
    Result<Borrow<T>> Resolve(ObjectHandle, GcReadScope&, const ObjectAccessContext&);
    Result<void> RequestDestroy(ObjectHandle, DestroyReason);
    ObjectStats SnapshotStats() const;
};
```

NewObject는 typed layout/TypeInfo 검증과 ConstructionContext를 사용한다. Resolve는 index 범위·generation·IsA·Published·cutoff eligibility·요청 ContextId를 검사한다. DestroyRequested 이후 public Resolve는 실패한다. context 없는 Core 객체도 허용하고 WorldId/Epoch는 후속 통합 계층에서 ContextId로 전달한다.

generation wrap 직전 슬롯은 영구 격리하고 추가 슬롯을 사용한다. 실패 시 reserved slot의 본체·root·type lease를 역순 해제한다. 디렉터리/slot 메모리의 주소 안정은 객체의 논리 생존과 다른 보장이다. direct delete와 사용자 stack Object 생성은 공개 API에서 차단한다.

11 참조·소유권·쓰기 장벽 API

11.1 사용자에게 보이는 타입

타입	소유·도달성	허용 연산
`RootRef<T>`	native root record를 통해 graph 보존	기존 lease copy/move·Resolve·Reset
`ObjectRef<T>`	도달 가능한 owner의 추적 edge	owner-aware TrySet·Clear·조회
`GcArray<ObjectRef<T>>`	owner가 정해진 edge 집합	TryAdd/TryReplace/Erase·trace
`WeakObjectRef<T>`	보존 없음	TryPin 또는 짧은 TryResolve
`Borrow<T>`	현재 read phase의 비소유 접근	scope 내 동기 사용만
NativeRef/UniquePtr	관리 heap 밖의 저장소 소유	GC edge로 사용 금지

표 8

RootRef의 최초 등록은 실패 가능한 TryPin/NewObject 경로로 수행한다. RootRecord는 native refcount와 handle을 보유하며 기존 RootRef 복제는 record count 증가로 처리하여 allocation 없는 noexcept 보존 이전을 가능하게 한다. 서로 다른 RootRef instance의 record 공유는 thread-safe지만 같은 C++ 변수의 동시 write는 허용하지 않는다. count wrap은 진단 후 실패/종료로 처리하며 0에서 부활시키지 않는다.

RootRecord 최초 등록·마지막 Reset의 unlink·GC snapshot 취득은 같은 root/GC gate에서 선형화한다. 초기 구현은 gate 아래에서 root handle을 복사하고 동일 safepoint에서 shade한다. record 주소를 gate 밖에서 순회하는 후속 구현은 scan lease가 필요하며 마지막 Reset 이후에도 해당 lease 종료까지 record 저장소를 반환하지 않는다. collector의 snapshot 보존은 일반 gameplay root 승격 권한이 아니다. CT14에 scan 취득/마지막 Reset/pin commit의 강제 interleaving을 포함한다.

```
template <class T>
Result<RootRef<T>> TryPin(WeakObjectRef<T>, ObjectAccessContext&);
template <class T>
Result<void> TrySet(ObjectWriteContext& owner, ObjectRef<T>& field, ObjectHandle target);
void Clear(ObjectWriteContext&, AnyObjectRef&) noexcept;
```

신규 handle 승격과 다른 owner 필드 연결은 generation/type/context/cutoff/상태 검사를 같은 GC gate 안에서 완료한다. 실패 시 이전 field를 유지한다. 이미 등록된 RootRef의 복사·이동은 DestroyRequested 저장소 보존만 전달하며 일반 논리 사용 권한을 주지 않는다. clear는 cleanup 권한으로 허용한다.

11.2 owner-aware edge

ObjectRef의 raw handle storage를 일반 public copy-assignment로 다른 owner에 복사하지 못하게 한다. generated accessor·GcArray가 owner identity를 알고 barrier를 호출한다. 같은 owner의 GcArray buffer 재배치는 기존 edge를 실패 없이 이전하는 내부 noexcept 연산이다. 다른 owner로의 clone은 remap·검증·barrier를 거친다.

ObjectRef를 stack/native Array에 복사하는 것으로 root가 생성되지는 않는다. MetaCompiler는 reflected Object raw pointer·비추적 reference container를 오류로 판정한다. visitor 없는 custom container는 managed field로 등록할 수 없다. VM stack/root provider는 후속 등록 계약을 사용하며 Foundation에서 VM을 구현할 필요는 없다.

11.3 Borrow와 read phase

Borrow는 safepoint·yield·frame 경계·callback 비동기 전달을 넘지 못한다. 필요한 수명은 RootRef로 보존하고 재개 후 다시 Resolve한다. GcReadScope는 reader의 완료를 추적하지만 도달성 root나 일반 field lock이 아니다. debug scope generation으로 만료된 Borrow 사용과 safepoint 진입을 검출한다. 장시간 reader를 도입하려면 별도 immutable snapshot 또는 graph 보호 계약을 추가한다.

Resolve와 TryPin에는 ObjectAccessContext를 명시 전달한다. ContextId와 epoch는 fake context만으로도 테스트하며 thread-local current World를 암묵 조회하지 않는다. context 없는 Core 객체에는 Unscoped 값을 사용하고, 이 값으로 scoped 객체의 검사를 우회하지 않는다. Borrow는 논리 호출 권한을 영구 보장하지 않는다. 사용자 callback/virtual extension 복귀 후 owner state/context를 재검증하고 DestroyRequested면 정리·반환만 수행한다. 이후 일반 task·activation·외부 gameplay commit을 금지하며 teardown 권한은 별도다. CT20에 callback이 자신의 owner를 파괴하는 fixture를 포함한다.

12 생성·기본값·복제 transaction

12.1 생성 경로

```
Validate Type / Budget
 → Reserve graph identities
 → NoSafepoint constructor
 → Register fully-constructed members in ConstructionContext
 → Apply class defaults / instance values
 → Deserialize + remap references
 → Validate + Initialize
 → Atomic graph publication
 → optional integration Register / BeginPlay
```

constructor는 bounded/no-yield/no-safepoint이며 임의 callback·IO·backend 등록을 수행하지 않는다. 실패 가능한 업무는 Result Initialize로 분리한다. 초기 메모리 부족은 진입 전 예산 확인 또는 실패로 처리하고 ctor 중 GC를 재진입하지 않는다.

ConstructionContext는 완전히 생성된 구성원과 외부 dependency를 private root로 유지한다. 부분 생성 객체는 visitor에 전달하지 않는다. 긴 defaults/fixup 단계 사이에는 이 root context를 유지한 채 safepoint를 허용한다. internal lookup과 public Resolve를 분리하고 public에는 graph commit 전 노출하지 않는다. 실패 시 생성 완료 역순 정리와 slot 반환을 수행한다.

publication 전 Initialize는 값 계산과 ConstructionContext journal이 소유하는 가역 자원만 허용한다. 외부 observer에 공개되는 delegate/task/backend 등록은 commit 후 integration이 수행한다. 사전 rollback은 context root를 유지하며 journal을 역순 해제하고, 진행 중 callback이 있으면 context/storage를 완료까지 보존한다. commit 이후 Register/BeginPlay 실패는 unpublished rollback으로 되돌리지 않는다. PublishedGraph owner가 RequestDestroy와 정상 retire를 수행하며 상위 결과는 commit 실패와 published integration 실패를 구분한다. CT15에서 두 실패 구간을 각각 주입한다. R1에서는 integration hook을 비활성으로 두어도 된다.

12.2 ClassDefaults·instancing

ClassDefaults는 TypeId/revision별 불변 archetype이다. 순서는 native default → class default → prefab/variant → instance override → 명시 spawn parameter다. Foundation은 class default와 instance override까지 테스트하고 prefab importer는 후속으로 연결한다. default subobject는 SubobjectGuid와 인스턴스별 clone/remap을 갖는다. 가변 component를 모든 instance가 공유하지 않는다.

```
Result<OwnedObjectGraph> AllocateObjectGraph(const GraphPlan&);
Result<void> ApplyDefaults(ConstructionContext&, const DefaultSet&);
Result<void> RemapReferences(ConstructionContext&, const RemapTable&);
Result<PublishedGraph> CommitObjectGraph(ConstructionContext&);
```

OwnedObjectGraph와 ConstructionContext는 move-only다. Commit은 context를 참조로 받아 먼저 검증하고 성공할 때만 ownership을 PublishedGraph로 이전한다. 실패하면 caller의 context에 cleanup ownership이 그대로 남으며 재시도 또는 rollback 할 수 있다. moved-from context가 실패 cleanup의 유일 정보를 잃는 API를 구현하지 않는다.

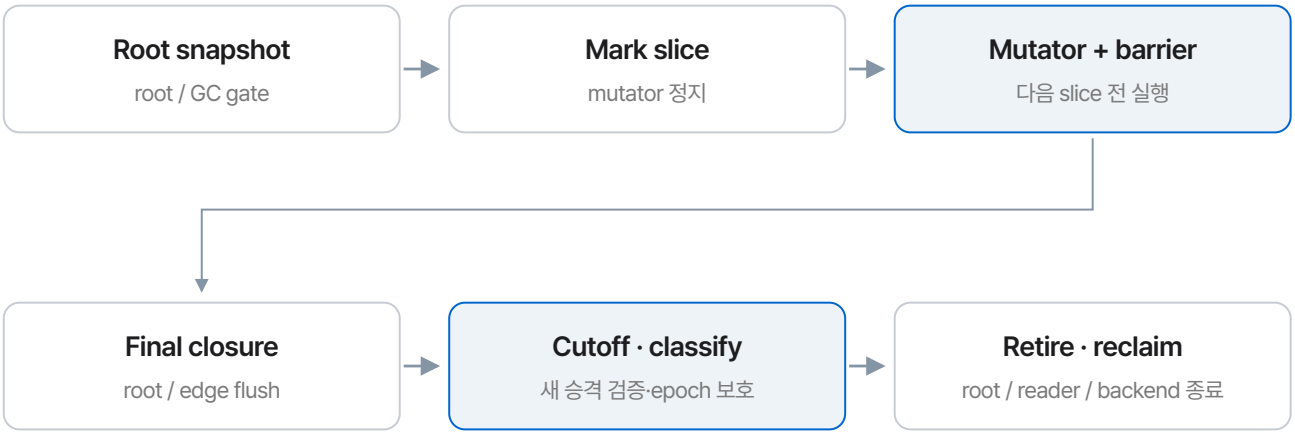
원본 GUID·Outer·transient/editor-only·soft external reference 복사 정책을 type/schema가 선언한다. 기본값/TypeInfo는 ModuleLease를 유지한다. 특정 World actor를 ClassDefaults가 강하게 보존하지 않게 검증한다.

13 GC의 단계별 구현과 수거 규약

13.1 첫 collector

R1은 단일 Game mutator·정지형 mark-and-sweep으로 시작한다. address nonmoving과 generated/manual visitor 경계를 처음부터 고정한다. root snapshot·iterative mark queue·weak 검사·retire record·budgeted cleanup을 분리하여 full collector가 이후 최적화의 oracle로 유지되게 한다. recursive C++ call stack으로 깊은 객체 graph를 순회하지 않는다.

R2는 barrier API·cutoff·phase 검증까지 포함한다. 병렬 marking은 mutator가 정지한 구간에서만 실행하며 incremental marking은 여러 slice 사이에 mutator를 재개한다. fully concurrent field scan은 필수 초기 범위가 아니다. Epic의 증분 GC 자료는 barrier 누락 위험의 참고이며 자체 알고리즘의 정확성을 대신 입증하지 않는다. D4



첫 구현: full STW → 점진 phase → 선택 병렬 mark

오수거 0건이 선행 조건 · 성능 목표는 측정 후 확정

그림 2 · 객체 수거의 단계와 초기 구현 순서 (개념도)

13.2 cycle와 cutoff

safepoint 협조 완료·Borrow 0 확인 후 epoch/queue를 초기화하고 roots를 스캔한다.
marker는 handle·field cursor로 graph를 순회한다. 병렬 worker는 test-and-set과 명시 work counter로 완료를 판정한다.
mutator 재개 중 신규 edge/root는 대상 shade, 신규 객체는 생존 표시와 outgoing trace를 함께 등록한다.
마지막 safepoint에서 root provider·thread-local barrier queue·신규 객체를 flush한다. 미완료 work가 있으면 다음 slice로 이월한다.
closure 확인과 cutoff publication을 신규 acquisition gate와 선형화한다. cutoff 대상은 실제 Retiring 전이 전에도 Resolve/TryPin/TrySet이 실패한다.
chunk별 후보를 Retiring으로 분류한다. 이전 분류 완료 전 mark bitmap/epoch를 재사용하지 않는다.
이미 분류된 retire record는 backend/read/metadata 수명을 유지하며 다음 mark와 공존할 수 있다. 이 hold는 GC self-root가 아니다.
born epoch로 sweep 도중 생성한 객체를 후보에서 제외한다. 64-bit epoch wrap은 정지·drain·재설정 경로로 처리한다. barrier queue가 포화하면 spill budget 또는 admission 제한을 사용하고 handle을 버리지 않는다. 계속 mutation이 발생하여 closure가 지연되면 loading full-cycle 또는 명시적인 작업 제한으로 전환한다.

13.3 점진 visitor의 cursor 무효화

큰 GcArray를 분할 스캔하는 도중 erase/reorder가 발생하면 unscanned edge가 cursor 앞쪽으로 이동할 수 있다. 삽입 barrier만으로 이 경우가 해결된다고 가정하지 않는다. TraceCursor는 owner handle·FieldId·container mutation version·index를 보관하며 raw element pointer를 slice 사이에 보관하지 않는다.

구조 변경은 version을 증가시키고 해당 owner/field의 rescan을 예약한다. resume 시 version이 다르면 해당 field를 처음부터 다시 스캔한다. 이미 marked owner라는 이유로 dirty rescan을 버리지 않는다. 참조 삽입·교체는 대상 shade도 수행한다. 초기 대안으로 bounded immutable reference snapshot을 사용할 수 있으나 그 메모리 비용을 별도 계측한다.

array 삭제·swap erase·reorder·rehash·resize를 slice 사이에 강제하고 full oracle의 도달 집합을 빠짐없이 포함하는지 확인한다. iterator 무효화 문제를 GC 정확성 fixture로 관리한다.

13.4 정확성과 성능의 분리

full oracle보다 작은 live set은 실패다. 보수적 잔존은 기록하고 root/변경이 제거된 후 후속 full cycle에서 해제되는지 확인한다. 성능 0.5 ms mark slice·1.0 ms safepoint p99는 기존 계획의 미측정 soft 목표로 유지한다. root 수·edge 수·worker 수·최대 정지·backlog를 함께 기록하며 목표 미달을 오수거로 보상하지 않는다.

14 파괴·종료·모듈 해제

14.1 논리 종료와 저장소 보존

RequestDestroy는 owner lane에서 DestroyRequested와 신규 일반 task/activation 차단을 먼저 설정한다. 기존 소유자의 짧은 teardown 권한으로 EndPlay/Deactivate hook을 실행하되 no-yield/no-safepoint를 지킨다. hook 재진입은 idempotent이며 public Resolve/TryPin은 계속 실패한다. 다른 lane은 command를 제출하고 owner commit 결과를 관찰한다.

BeginDestroy는 reader가 종료된 경계에서 1회 실행한다. managed 이웃의 생존에 의존하지 않고 자기 edge clear·독립 backend token 정리를 수행한다. 후속 callback에는 `Object*` 를 캡처하지 않는다. 기존 RootRef 및 도달 가능한 ObjectRef edge는 저장소를 보존하며 남은 field는 계속 trace한다.

최종 회수는 cleanup ready AND strong reachability 없음 AND reader 종료 AND backend/task 완료다. TypeInfo/module code도 destructor/visitor 사용이 끝날 때까지 유지한다. 객체 메모리가 남아 있다는 사실로 일반 게임 호출을 허용하지 않는다. retained bytes·destroy age·root chain을 출력하고 오래 유지된 root는 원인을 수정하며 timeout 강제 free는 금지한다.

14.2 boot·shutdown의 실제 순서

```
Boot: PlatformBase → Core allocator/diagnostic → Task/IO/Trace
      → Schema/Archive → ObjectRuntime/manual types → TestHost
Stop: normal admission close → owner cancel/deactivate
      → cleanup-only tasks → callbacks/readers/backend drain
      → roots release + final collection
      → GC가 생성한 cleanup/retire/backend completion 추가 drain
      → owner/retire/reader/lease closure 확인 → object/type ops release
      → ObjectRuntime → IO/Task join
      → Trace drain/flush + writer join → Core/Platform shutdown
```

위 순서는 R2 전체 구성 기준이다. R1은 Task inline executor·C06 logging/최소 CPU trace·R1 counter/report를 필수 조립한다. 비동기 IO·전체 Trace worker·Archive는 생략할 수 있다. ObjectSerialization은 C23에서 추가하며 ObjectRuntime은 이를 역참조하지 않는다.

최종 GC 이전에 type table·allocator·scheduler가 사라지지 않아야 한다. 종료 중 필요한 cleanup task는 제한된 capability로 승인한다. native module unload는 task·callback·object·archetype·TypeInfo·script frame lease가 0인 경우에만 실행한다. 현재 R1/R2는 static link·process restart로 충분하며 dynamic unload의 구현을 Foundation의 선행 조건으로 강제하지 않는다. 수명 규약은 처음부터 유지한다.

15 Core 구현 순서와 첫 실행 경로

15.1 순서와 의존 조건

작업 ID는 착수 후보를 식별한다. 숫자 순서만으로 병렬 착수하지 않고 작업 CSV의 선행 ID를 따른다. 각 작업은 해당 테스트와 진단 출력을 포함한 작은 변경으로 완료한다. 테스트 ID는 묶음이며 초기 단계는 지원된 부분만 실행한다. C07의 CT10은 수동 schema만, C06의 CT08은 동기 executor만 검증한다. generated 비교는 C20, N-worker/취소 경합은 C17, 전체 묶음은 C26에서 요구한다. 미실행 부분을 PASS로 합산하지 않는다. 초기 객체 fixture에는 문자열 이름 대신 고정 Guid를 사용하고, GC는 단일 실행 lane에서 수행하여 고급 Core 기능의 완료를 기다리지 않는다.

Gate	작업	실제 완료 결과
R0	C00–C06	Win/Mac scaffold/runner·N-fail·Array·동기 Task·초기 log/profile·하네스 자체 검사
R1	C07–C12	순환 graph·root 제거/full GC·객체/GC/heap ledger·원인 report·종료 closure
R2-A	C13–C20	컨테이너/text·선정 math adapter/Core 시간·Task/IO·trace/analyzer/viewer·Archive·메타
R2-B	C21–C26	managed container·점진 phase·복제/migration·Win/Mac 필수 하네스·viewer/report·CF01–12
선택 최적화	C27–C31	동적 확장 loader·allocator·병렬 GC·성능·빌드 cache

표 9

C06에서 move-only Function과 loader 비의존 ModuleLease를 먼저 도입한다. C14는 Function의 SBO 경계 회귀 검사를 확장한다. R0의 Task 계약은 동일한 Submit/terminal 의미를 갖는 TestInlineExecutor로 시작한다. ObjectRuntime은 scheduler facade에 의존하며 실제 worker pool은 C17에서 주입한다. inline 실행은 GC의 병렬성 검증으로 인정하지 않는다. C02에는 Registry에 필요한 atomic과 mutex의 최소 구현을 포함하고, C16에서 thread·condition·wait와 memory-order 검증을 확장한다. 예정된 의존 모듈을 빈 성공 stub으로 연결하지 않는다.

15.2 초기 4주 제안

아래 일정은 Foundation 담당 3–5명 가정의 계획 예시이며 확정 납기나 측정된 생산성이 아니다. R1 통과 후 추정치를 다시 산정한다. 모든 인력이 같은 public header를 동시에 변경하지 않도록 API 담당자를 둔다.

구간	우선 결과	병행 가능 작업	중단·재설계 조건
1주	C00–C04: CPU 실행·primitive·실패 처리·allocator	include audit·test fixture 규약	STL 경계/allocator ownership 불명확
2주	C05–C08: Array·runner·수동 타입·registry	문서화된 fake input event 모델	재할당 수명·slot ABA 실패
3주	C09–C11: 참조·생성·full GC	CT stress seed와 graph oracle	root/weak 경합·오수거

구간	우선 결과	병행 가능 작업	중단·재설계 조건
4주	C12·R1 통합: destroy·shutdown	후속 Task/Archive 계약 검토	종료 deadlock·leak·type 조기 해제

표 10

첫 실행은 FoundationTests에서 LeafObject, PairObject, ArrayOwner를 생성하여 strong cycle을 구성하고, RootRef를 제거한 뒤 full GC로 모두 회수하는 경로다. stale WeakRef, 실패한 생성, DestroyRequested인 root 보유 객체, 반복 boot/shutdown을 같은 runner에서 검증한다. 화면·에셋·게임 모듈은 필요하지 않다.

15.3 작업 완료 규칙

모든 구현 단위는 public API·오류/thread/lifetime 계약과 unit/contract/fault/concurrency 하네스, 독립 oracle·negative control, 종료 후 자원 ledger·로그/trace/report를 함께 제출한다. 단일 thread 설계도 진입 lane/공유 mutation 금지의 검증을 남긴다. 계획 CSV의 검증·장애주입·동시성검증·완료증거를 각 구현 PR에 연결한다. 미지원 상태는 추적 ID와 capability 설명으로 남길 수 있다. 테스트가 없는 고급 최적화를 완료율에 포함하지 않는다. R2의 기간은 기존 전체 계획의 Foundation 6개월 가정 안에서 재산정하며 C27–C31 전체를 R2 선행 조건으로 추가하지 않는다.

15.4 Core 작업별 산출물

ID	Gate·선행 작업	작업·산출물	검증
C00	R0 / 없음	저장소·빌드·scaffold — Win/Mac CPU-only·STL/include audit·최소 module/test scaffold·spaces4/clang-format·가독성 규약	CT01;TT01
C01	R0 / C00	Base·ABI 타입 — Guid POD·compiler macro·checked size 규약	CT01;CT02
C02	R0 / C01	최소 Platform primitive — Win/Mac page·clock·atomic·mutex 및 Public/Platform/Sync.h·fatal 출력	CT02;CT07
C03	R0 / C01	Result·값 수명 보조 — Error/Result·traits·move·명시 실패	CT02;CT04
C04	R0 / C02;C03	기준 allocator — simple heap·allocation tag·N-fail·leak ledger	CT02;CT03
C05	R0 / C04	최소 Array·Span — move-only·self-alias·overflow·실패 보존	CT04
C06	R0 / C05	CPU runner·초기 로그·하네스 — Function·ModuleLease·inline executor·bounded log/CPU scope·TestClock/N-fail helpers	CT01;CT08(동기);CT24(R0);HT01–06(R0)
C07	R1 / C06	수동 schema·RuntimeTypeOps — 고정 Guid 타입·typed visitor·등록 검증·type/object ledger	CT10(수동);CT24(R1)
C08	R1 / C07	청크 Registry — slot generation·gate Resolve·publication·객체/slot counter	CT13;CT24(R1)
C09	R1 / C08	Root·Weak·Borrow — root record·context·scan lease·pin cutoff·root 진단	CT14;CT24(R1)
C10	R1 / C09	객체 생성 transaction — private context·rollback/commit·실패 단계 ledger	CT15;NL01
C11	R1 / C10	full STW collector — 단일 lane graph mark/sweep·독립 oracle·GC phase/report	CT17;CT24(R1);NL02
C12	R1 / C11	논리 파괴·종료·누수 Gate — destroy admission·retire·backend fake·owner/root/heap 잔존 검사	CT20;CT21;CT22;NL01–03;NL06

ID	Gate·선행 작업	작업·산출물	검증
C13	R2-A / C05	문자열·Name·Guid·Hash — UTF-8·intern·지속 ID·hash 정책	CT06
C14	R2-A / C06	Core container 확장 — InlineArray/HashMap/BitArray/SlotMap·Function 검증	CT05
C15	R2-A / C02;C05;C06	검증된 수학 adapter·Core 시간 — Rocks Vec3/Quat/Mat4·math 후보 선정·x86/ARM64 SIMD·Transform·ClockService/StepPlanner	CT06;MX01–08;TK01–04
C16	R2-A / C02;C06	thread·sync 확장 — C02 Sync.h 확장·thread lifecycle·wait·memory order fixture	CT07
C17	R2-A / C14;C16	Task scheduler — worker DAG·cancel·terminal·shutdown·직렬/병렬 의미 비교	CT08;TK05–06
C18	R2-A / C13;C17	비동기 IO·필수 수집·분석·viewer — offset IO·lease·loss-aware trace·headless inspect/report·offline timeline 초판	CT09;CT24;OB01–06(CPU)
C19	R2-A / C07;C13;C14	Archive·resolver — bounded TLV·stable field ID·reference resolver	CT11
C20	R2-A / C19	Clang 메타 생성기 — AST schema·trace 생성·재생성 최소화	CT10;CT12
C21	R2-B / C12;C14;C20	managed field/container — owner-aware edge·GcArray·구조 mutation version	CT18
C22	R2-B / C17;C21	점진 GC phase — barrier·root flush·cutoff·epoch·bounded queue	CT18;CT19
C23	R2-B / C19;C21	기본값·복제 — ClassDefaults·SubobjectGuid·cycle remap·Archive bridge	CT16
C24	R2-B / C18;C22;C23	수명 통합 — task/IO/backend/type lease·재진입·정상 retire·scope ledger	CT20;CT21;CT22;NL03;NL06–08
C25	R2-B / C20;C23	schema migration — unknown field·version·golden fixtures	CT11;CT16
C26	R2-B / C15;C24;C25	Foundation 필수 Gate — CF01–12·Win/Mac·누수/하네스·CPU/Task/GC/memory viewer·report	CT01–24;MX01–08;TK01–06;NL01–03/06–08(CPU);HT01–06;OB01–06(CPU)
C27	선택 R3 / C26	Host 확장 loader — C ABI 검증·registration transaction·Busy unload	CT22;CT23
C28	선택 R3 / C26	allocator 최적화 — slab·thread cache·remote free·arena lease	CT02;CT03
C29	선택 R3 / C26	병렬 mark — GC 전용 진척·parallel work counter·oracle 비교	CT17;CT18;CT19
C30	선택 R3 / C26	계측 비용·분석 성능 최적화 — sampling/압축·대형 trace 탐색·수집 overhead 미세조정만	CT24;NL08
C31	선택 R3 / C26	빌드 반복 최적화 — action manifest·cache key·Rocks Build/분산 adapter 경계	CT01;CT12

표 11

16 Core·Object 검증 기준

16.1 Foundation Gate

CF 항목은 이번 상세안의 검증 묶음이다. 전체 계획의 Core Gate 취지를 구체화하며 기능 개수와 별도로 판정한다. CTest 또는 동등 runner가 machine-readable 결과·seed·toolchain·commit·config를 저장한다. 다음 수치들은 제안된 fixture 규모이며 측정 결과가 아니다.

Gate	검증	연결 테스트·통과 기준
CF01	독립성	CT01·CT23: World/Window/GPU/VM 없이 Core/Object CPU-only build·실행
CF02	정책	CT01: STL/예외/RTTI·공개 header·allocator/SDK 경계, 포맷 규칙
CF03	메모리	CT02–03: 정렬·overflow·N-fail·교차 thread free·누수/소유권 오류 0건
CF04	컨테이너·값 타입	CT04–06·MX01–08: nontrivial 수명·이동·self-alias·iterator·Transform·수학 adapter
CF05	Task	CT07–09: publication·의존·취소·terminal 1회·IO/queue 수명·진척
CF06	Registry·생성/파괴	CT13·CT15·CT20–21: generation·publish·rollback/정상 retire·오래된 handle 차단
CF07	참조	CT14·CT20·CT23: Root/Object/Weak·Borrow·context·신규 취득 cutoff·논리 종료와 저장소 보존
CF08	GC	CT17–19: 독립 oracle·barrier·rescan·cutoff·epoch·오수거 0건
CF09	기본값/복제	CT15–16: subobject 독립·cycle/shared child remap·stable ID·기본값
CF10	메타/Archive	CT10–12: 수동/생성 schema·3개 version migration·bounded archive·unknown field
CF11	진단	CT24: CPU/Task/GC/memory·root chain·loss/truncated·headless capture→offline viewer/report
CF12	통합	Windows/macOS에서 CT01–24의 필수 subset·지원 sanitizer·TK01–06, CT22 반복 shutdown·모든 owner/lease closure

표 12

전체안 1.2절의 CF ID·의미를 그대로 사용한다. CT는 구체 fixture 식별자이며 별도 gate 의미로 CF 번호를 재정의하지 않는다. Linux CPU suite는 M3 필수 후속, Linux 서버 통합은 M5 gate로 관리한다.

16.2 테스트 fixture 상세

ID	입력·실행 방법	검증 oracle
CT01	각 Public header를 빈 TU로 컴파일; 전체 target graph 검사	상위 include/SDK/STL 유입·dependency cycle 없음
CT02	0/1/정렬 경계/페이지 경계/최대 size; invalid alignment·overflow	정상 정렬, 오류 구분, 기존 상태 보존
CT03	allocator N-fail sweep; worker에서 할당·다른 lane 해제·thread exit	allocation ledger와 destructor 횟수, remote free 누락 없음
CT04	move-only probe Array; reserve/self-append/self-push/remove	소유 자원 총수 보존, 실패 시 관찰 상태 불변
CT05	hash collision/tombstone/re-hash; SlotMap 재사용; Function SBO 경계	단순 reference model과 lookup·erase 결과 일치
CT06	잘못된 UTF-8·Name hash collision·Guid fixture·affine/shear	명시 오류·canonical serialization·검증된 backend/독립 golden·x86/ARM64 tolerance
CT07	atomic publication litmus·mutex·wake·timeout·shutdown	미초기화 관측 없음; supported sanitizer 결과
CT08	0/1/N worker·cancel/complete·완료 handle 1,000개 보관	terminal/capture 소멸 각 1회, body 중 소멸 0, payload drain 후 capture root 0
CT09	긴 IO·partial read·cancel·late completion·trace full	buffer lease 보존, stale callback 차단, loss counter
CT10	수동/generated 대조·등록 source arena reset/poison·old reader 중 unregister	TypeId/FieldId/trace 일치, dangling metadata 0, Busy/revision lease 보존
CT11	TLV golden bytes·unknown field·version migration·잘못된 길이	bounds 초과 없음, migration 결과 또는 명시 실패
CT12	compiler 진단·formatter/config 변경·생성 파일 미변경 rebuild	stale hit 0, producer 1개, 동일 최종 출력 timestamp·hash 유지
CT13	64K slot 전후 생성·reuse; 축소 generation test build	구세대 handle 거부, wrap slot quarantine
CT14	TryPin과 cutoff/destroy/reuse interleaving	성공한 pin은 storage 보존, 실패는 신규 root 미생성
CT15	constructor/default/field fixup/Initialize 각 단계 실패	publish 0건, 역순 rollback, context root 누락 없음
CT16	clone의 cycle·shared child·subobject remap·외부 weak	graph topology·참조 정책·stable field 의미 보존
CT17	seed별 1,000–100,000 객체 directed graph	독립 full DFS live set의 부분집합을 회수하지 않음
CT18	slice 사이 edge 삽입·root 취득·배열 reorder·swap-remove	incremental live set ⊇ full oracle live set
CT19	allocation during GC·queue 포화·chunk 분류 중 새 cycle 요청	epoch 오염 없음, barrier 유실 없음, 안전한 backpressure
CT20	DestroyRequested를 RootRef로 장시간 보유	Resolve 실패, destructor 지연, root 제거 후 회수
CT21	fake GPU/audio/IO token 지연·중복·뒤늦은 ACK	최초 완료 이후 중복 무해, 유효 lease 이전 free 없음
CT22	shutdown 중 queued callback·type unregister·재등록	정해진 drain 순서, Busy 보고, 마지막 destructor 이전 code 유지
CT23	TestObjectModule·TestAllocator를 별도 target으로 추가	Core source diff 0건, 올바른 registration·에러 반환
CT24	R0 log/scope·R1 counter/root chain; R2 queue 포화·allocator 재귀·metadata unload·shutdown/crash·viewer	단계별 필수 수집·손실/truncated·baseline 표시·원인 report·동일 seed 재현·overhead 보고

표 13

ASan/UBSan/TSan은 toolchain·platform 지원에 따라 별도 job으로 실행한다. 실행하지 못한 sanitizer를 통과로 기록하지 않는다. 특히 TSan 실행 환경은 지원 조합을 고정하고 전체 플랫폼을 동일하게 지원한다고 가정하지 않는다. fixture·sanitizer·stress는 정확성 근거를 보완하며 C++ data race 부재의 형식 증명을 대신하지 않는다. D14

PR에서는 짧은 deterministic suite, 야간에는 seed 확장·N-fail·장시간 생성/해제, release 후보에서는 모든 지원 target 조합과 rollback fixture를 실행한다. 잠정 기준은 PR suite 5분 이내·야간 stress 2시간으로 두고 첫 baseline 이후 조정한다. 시간 초과를 해결하려고 오류 fixture를 제거하지 않는다.

16.3 모든 구현의 하네스와 관측 수용 기준

C00/C06부터 TestClock·DeterministicExecutor·NFailAllocator·GoldenFixture를 도입하고 C18의 ScriptedIo, C24의 FakeBackendCompletion으로 확장한다. 하네스는 오류·지연·재정렬·중복 completion을 주입하며 실제 OS/장치 시험을 대체하지 않는다. 독립 reference model·수동 golden·명세에서 기대값을 산출한다. barrier 누락·잘못된 generation·중복 callback·조기 free·한 tick 입력 중복을 의도적으로 삽입한 negative control이 반드시 실패해야 한다.

ID	필수 증거
HT01–02	작업/파일→계약→test→실행 결과 추적; unit/contract/상태 전이·독립 oracle
HT03–04	fault/negative control의 실패 검출; seed/clock/schedule 재현
HT05–06	Win/Mac·fake/실장비·지원 sanitizer의 PASS/FAIL/NOT RUN 분리; 테스트 자체 task/root/allocation/handle closure
NL01–03	생성/등록 N-fail rollback; root/weak/cutoff 수명; 늦은 완료·module lease 안전
NL06–08	정상 shutdown 잔존 0·crash 불완전 표시; ledger/sanitizer 교차; 최적화/worker 수 변경 전후 소유권 동일
OB01–02	log 필터/burst/rotation·CPU/Task/GC/memory capture→읽을 수 있는 viewer/report
OB03–04	queue loss·partial trace·metadata unload·중간 capture baseline; allocator/임계 구간 재귀·대기 없음
OB05–06	shutdown/hang/crash bounded flush·truncated 표시; headless/live/offline schema/clock·두 capture 비교

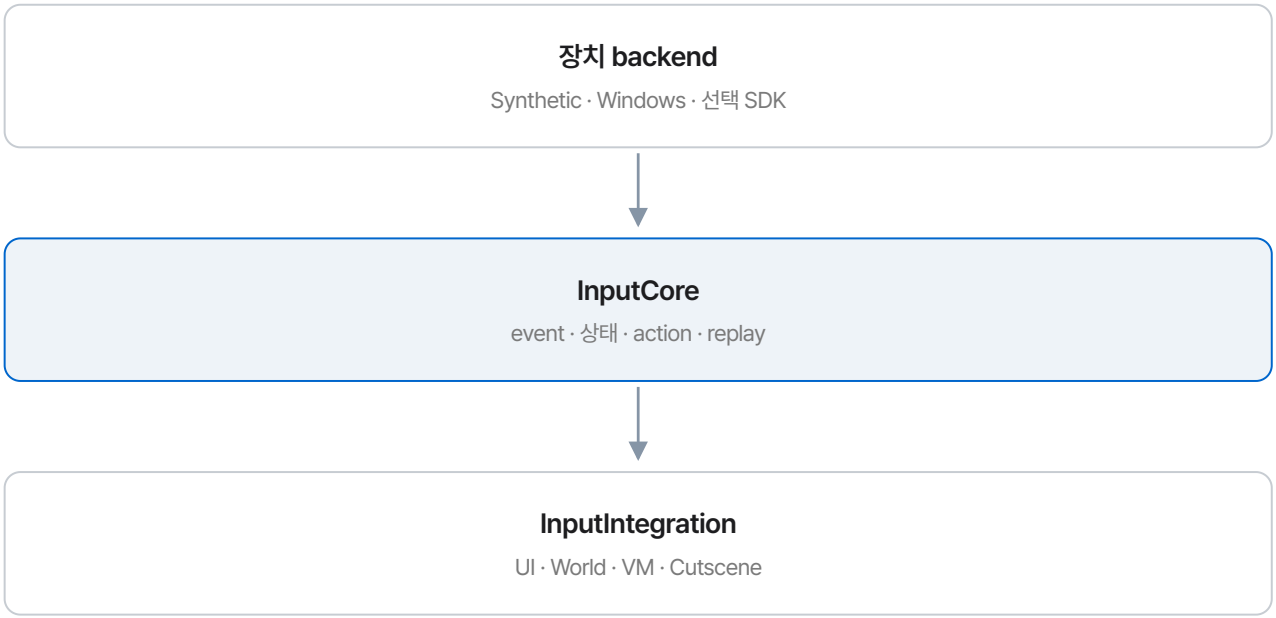
표 14

NL04의 PIE/streaming·NL05의 GPU device loss와 GPU 관측은 26절의 상위 시스템 최초 Gate에서 실제 backend로 확장한다. CPU fake 통과로 그 항목을 완료 처리하지 않는다. PR은 작은 deterministic suite, 야간은 seed/fault/stress/sanitizer, release는 두 필수 OS와 지원 장치 matrix를 실행한다. flaky 재시도 통과로 원인을 숨기지 않으며 quarantine된 필수 검사는 미완료로 집계한다. 영향 suite 선택·병렬 실행으로 시간을 줄이되 오류 경로를 삭제하지 않는다.

17 입력 시스템 경계와 확장 계약

17.1 단계와 데이터 흐름

입력은 OS event 전달, 정규화된 장치 상태, action 해석, 게임/UI 소비의 네 층으로 분리한다. InputCore의 테스트는 window·controller SDK·Object 없이 실행한다. 실제 사용자/계정·World·위젯 ownership은 InputIntegration과 Host에서 연결한다.



장치 callback: queue만 기록 · 게임/위젯 호출은 지정 phase에서 수행

generation · stream / route epoch · capability · owner lease

그림 3 · 플랫폼 입력과 게임 연결의 분리 (개념도)

계약	핵심 책임	확장 방법
InputBackend	장치 열거·capability·event 생산·output 요청	backend factory descriptor 등록
InputSystem	device generation·clock·queue·tick snapshot	allocator·clock·backend 목록 주입
ActionEvaluator	binding·processor·trigger·context 평가	typed processor/trigger descriptor 등록
InputRouter	focus·local user·UI/game priority	Host의 route policy 주입
InputIntegration	객체 callback·VM event·network command	weak owner+subscription token

표 15

InputBackend는 Windows GameInput/RawInput, macOS AppKit/GameController, 선택 SDL/Steam/콘솔 계열처럼 독립 target으로 추가한다. vendor ID enum을 InputCore에 추가하지 않고 BackendId/FeatureId Guid와 구조화된 capability를 사용한다. 필수 표준 control 의미는 버전 있는 schema로 정한다. 새 vendor 기능은 확장 capability payload로 전달하되 크기·version·owner lease를 검사한다.

17.2 핵심 API

```
struct DeviceHandle {
    uint32 slot, generation;
};

struct BackendId {
    Guid value;
};

struct InputCapabilities {
    DeviceClass deviceClass;
    CapabilityBits standard;
    Span<const FeatureDescriptor> extensions;
};

struct InputBackendDesc {
    ExtensionHeader header;
    BackendId id;
    InputBackendOps ops;
};

class InputSystem {
public:
    Result<BackendRegistration> RegisterInputBackend(const InputBackendDesc&, ModuleLease);
    Result<DeviceSnapshot> QueryDevice(DeviceHandle, FrameArena&);
    Result<void> AdvanceInput(const InputTick&);
    Result<ActionSnapshot> QueryAction(const InputReadKey&, ActionId) const;
    Result<ActionEventBatch> CopyActionEvents(const InputReadKey&, Allocator&) const;
    Result<void> PumpDevicesAndUi(MonotonicTime now);
    Result<OutputTicket> SubmitHaptics(const HapticRequest&);
};
```

예시 타입은 계약을 표시하며 완성된 header가 아니다. QueryDevice의 반환 span은 전달된 arena epoch까지만 유효하다. backend 등록 descriptor는 runtime이 필요한 값을 복사하고 function pointer에는 module lease를 유지한다. hot path virtual 호출은 장치 경계에 한정하고 action batch 평가는 compiled data로 수행한다. global singleton Input::Get()은 sample facade 이외의 기본 의존 방식으로 사용하지 않는다.

ActionSnapshot은 value·active 상태·tick/route/profile version을 담은 소유 값이며 transition 전달을 대체하지 않는다. ActionEventBatch는 같은 tick 안의 모든 phase 전이를 안정 순서로 보존하는 caller-owned 복사본이다. 보존 기간이 지난 tick은 HistoryExpired를 반환한다. 후속 zero-copy API에는 명시 ReadScope/epoch를 추가한다. PumpDevicesAndUi는 simulation tick 유무와 독립적으로 실행하므로 pause 중에도 text·UI repeat를 처리한다. 각 update domain은 승인된 route에서 event를 1회 소비하며 IT02/03에서 phase 수와 pause/resume을 검증한다.

InputBackend의 Start/Stop/Poll/SubmitOutput는 versioned ops table로 표현한다. Stop은 신규 callback admission을 닫고 진행 중 callback과 output ticket을 drain한다. 반환 이후 callback이 발생하지 않는다는 SDK 보장이 없으면 adapter-owned tombstone context를 마지막 callback lease까지 유지한다. 미지원 output은 Unsupported, 재연결로 무효화된 장치는 DeviceGone, 정지 중 요청은 ShuttingDown을 반환한다.

17.3 지원 상태와 fallback

상태	의미	소비자 동작
Supported	해당 조합에서 구현·검증 완료	선언된 capability만 사용
Partial	기본 입력만 지원, 일부 센서/output 미지원	기능별 capability 조회
Unsupported	현재 구현 없음	UI 숨김/대체 조작, 진단 사유 표시
Unavailable	구현은 있으나 SDK·장치·권한 등 사용 조건 없음	재시도 조건 표시, 정상 입력으로 위장 금지
Failed	사용 중 오류	취소·neutral·복구 또는 비활성화

표 16

이 문서의 실제 backend 상태는 모두 계획 단계다. I0는 Synthetic만으로 완료 가능하다. 미지원 플랫폼 때문에 Core를 변경하거나 공통 API에 해당 SDK의 placeholder 타입을 추가하지 않는다. SDK 업데이트와 장치 모델 추가는 adapter conformance suite로 검증한다.

18 입력 이벤트·시간·동시성

18.1 event 형식과 소유권

```
struct InputEventHeader {
    DeviceHandle device;
    uint64 sourceSequence;
    uint64 hostReceiveSequence;
    uint64 streamEpoch;
    InputTimestamp time;
    WindowEpoch window;
    InputRouteToken route;
    InputEventKind kind;
    uint32 payloadBytes;
};

struct InputConsumerScope {
    Guid id;
    uint64 generation;
};

struct InputReadKey {
    InputConsumerScope scope;
    LocalUserId user;
    InputUpdateDomain domain;
    uint64 clockEpoch;
    uint64 tick;
};

struct InputTick {
    InputReadKey key;
    MonotonicTime begin, end;
    InputRouteEpoch route;
};
```

InputTimestamp에는 정규화된 monotonic 값, source clock domain, quality(Source/Estimated/ReceiveOnly)를 기록한다. 모든 backend가 실제 장치 발생 시각을 제공한다고 가정하지 않는다. clock 변환 오차·불연속은 진단 값으로 남기고 동일 timestamp는 backend·device·sequence의 안정 순서로 정렬한다. callback thread는 검증된 POD/소유 payload만 queue에 추가하고 게임 객체·위젯을 직접 호출하지 않는다.

문자열·touch 배열은 bounded payload block lease로 소유한다. SDK event pointer나 stack buffer를 저장하지 않는다. Input lane이 queue를 drain하여 state/action snapshot을 publish하고 소비자는 immutable frame/tick snapshot을 읽는다. snapshot 교체와 storage 재사용에는 reader epoch 또는 복사 소유권을 사용한다.

Host가 routing admission 때 opaque InputRouteToken(routeId/generation)과 user/session epoch를 기록하고 focus 전환을 같은 route stream의 경계로 선형화한다. 아직 route를 정하지 않은 raw 수집 event에는 미지정 상태를 사용하며, 과거 event를 새 route에 재해석하지 않는다. 구 route의 key-up은 physical state 정리에만 사용할 수 있다. TextEvent에는 TextSessionId/generation과 composition/commit sequence를 추가하고 종료된 session의 늦은 commit을 거부한다. IT19는 같은 WindowEpoch의 text A/B 전환·PIE 재생성·지연 callback을 검사한다.

공유 물리 DeviceHub와 action 소비 scope를 분리한다. Host는 opaque InputConsumerScope를 session·route generation에 연결하고 종료 시 admission을 닫는다. InputCore에는 World/Widget 타입을 도입하지 않는다. AdvanceInput과 QueryAction/CopyActionEvents는 InputReadKey의 scope/generation·user·update domain·clock epoch·tick을 같은 history key로 사용한다. UIRealTime·gameplay fixed/frame·replay 영역의 같은 숫자 tick을 합치지 않는다. route 전환·재시작에는 새 scope generation 또는 clock epoch를 사용하고 기존 key를 재해석하지 않는다. 종료 scope에는 ScopeExpired, 보존 구간 밖에는 HistoryExpired를 반환하며 다른 scope의 최신 결과를 대체 반환하지 않는다. 종료 시 cancel batch의 소비·명시 폐기까지 기록한 뒤 scope를 회수하며, 지연 조회자가 소멸 scope의 terminal event를 받지 못하는 경우에도 expired 오류를 명시한다.

18.2 frame과 고정 tick

render frame 사이 press/release가 모두 발생한 경우에도 두 transition을 보존한다. 최종 down 상태만 비교하는 구현은 허용하지 않는다. 하나의 물리 event는 정해진 simulation tick에 한 번 배정하고 catch-up tick마다 같은 mouse delta를 반복 적용하지 않는다. axis 지속 상태는 다음 변경까지 유지하지만 edge와 상대 이동량은 해당 tick만 소비한다.

이 보존 계약은 엔진이 실제 수신한 transition에 적용한다. backend capability에 event stream/history 또는 state polling과 시각 품질을 명시한다. polling 사이 발생한 관측 불가 tap을 복원하였다고 보고하지 않는다. 해당 조합은 Partial과 제한으로 공개하며 사용 목적에 필요한 충실도를 지원 matrix에 표시한다.

늦게 도착한 event는 기본 정책상 다음 처리 tick에 넣고 Late 플래그를 기록한다. 로컬 입력만으로 과거 simulation을 자동 rollback하지 않는다. 시간 정밀도 정책은 replay header에 저장한다. pause 중 UI/장치 상태는 계속 처리하고 gameplay에는 cancel/neutral을 적용하며, resume 시 오래된 edge를 한꺼번에 전달하지 않는다.

18.3 bounded queue의 손실 복구

mouse motion은 버튼·wheel·focus·device 경계 사이에서만 합산할 수 있다. wheel 분수와 방향 반전이 trigger 의미에 영향을 주는 경우 합산 금지 정책을 선택한다. raw diagnostic replay 모드에서는 원본 motion 유지 여부와 손실을 명시한다.

critical queue도 유한하므로 절대 무손실이라고 선언하지 않는다. 별도 per-device generation mailbox에 NeedsResync와 최신 connection/focus epoch를 기록한다. edge 손실 감지 시 영향받는 active action을 Cancelled 처리하고 neutral 상태로 전환하며, 장치 snapshot을 다시 수집한 뒤 이미 눌린 control은 release 확인까지 억제한다. InputGap을 trace/replay에 기록한다. release를 추정하여 Completed/Triggered를 발생시키지 않는다.

복구 경계는 per-device InputStreamEpoch·lossVersion·producer sequence watermark로 표현한다. 구 epoch event를 폐기하고 adapter snapshot과 event 재개 경계를 선형화한 뒤 새 epoch를 승인한다. snapshot 미지원 provider는 정확 복구로 표시하지 않고 새 epoch에서 release/neutral을 관측할 때까지 억제한다. mailbox ACK는 처리한 lossVersion까지만 반영하며 복구 중 발생한 새 loss를

지우지 않는다. transient edge/motion은 snapshot에서 복원하지 않는다. IT04에 snapshot 전후 event·재손실·disconnect 경합을 추가한다.

text commit은 일반 motion과 함께 폐기하지 않는다. payload admission 실패 시 composition을 취소하고 입력 서비스/UI에 복구 필요 상태를 전달한다. 손실 후 중복 commit을 재구성하지 않는다. 개인정보 text는 기본 trace/replay 대상에서 제외하고 password/secure field는 항상 제외한다.

초기 queue 예산 예시는 장치 8개·8 kHz event 2개·최대 stall 100 ms를 포함한 synthetic burst로 산정한다. 64-byte event 4,096개는 약 256 KiB의 header 공간만 의미하며 payload·index·reserve를 별도 계상한다. 이 수치는 보장 용량이 아니며 high-water/loss/latency 계측으로 조정한다.

19 키보드·마우스·문자 입력

19.1 Windows·macOS provider 선택

Windows의 우선 backend 후보는 GameInput이다. 공식 문서는 여러 입력 장치의 공통 API와 PC/Xbox 경로를 설명하지만, 실제 지원 모델·기능은 고정한 SDK와 장비에서 검증한다. keyboard/mouse에는 RawInput 대체 backend를 제공할 수 있다. 동일 물리 장치·동일 channel의 owner provider는 하나만 지정한다. GameInput·RawInput·legacy window event를 동시에 game action으로 처리하지 않는다. D5, D6, D7

OS의 여러 API가 동일 장치를 노출하는 경우 DeviceIdentityResolver가 가능한 식별 정보를 비교한다. 불확실한 경우 임의 병합하지 않고 provider 선택 정책과 진단을 제공한다. 장치 제거 후 같은 slot이 재사용되어도 generation은 달라야 한다. 사용자 지정 이름·모델 GUID·접속 instance·지속 식별자를 같은 값으로 취급하지 않는다.

macOS는 AppKit window event의 keyboard/mouse와 GameController의 검증된 controller 경로를 별도 private adapter로 제공한다. 동일 physical/control channel의 owner provider는 하나이며 AppKit과 GameController가 같은 입력을 노출해도 action으로 중복 전달하지 않는다. AppKit/window/text 조작은 main-thread 제약을 유지하고 장치 callback은 POD/소유 payload를 queue로 전달한다. callback 종료·notification 해제·ObjC 객체 보유의 마지막 완료를 확인한 뒤 adapter storage를 반환한다. D15, D16

19.2 키보드의 세 가지 의미

채널	목적	규약
PhysicalKey	이동·단축키·재바인딩	scan code/location을 표준 control로 변환; 좌우 modifier 구분
LogicalKey	현재 배열의 의미 있는 key·메뉴 shortcut	layout 변경에 따라 갱신, physical과 구분
TextEvent	채팅·이름·문서 편집	IME composition/commit·selection·caret 처리

표 17

게임 key repeat는 장치 down edge와 분리한다. UI navigation repeat는 초기 지연·반복 간격을 설정하고 frame rate에 의존하지 않는다. 여러 키보드를 단일 seat로 합치는 기본 정책과 장치별 분리 정책을 Host에서 선택한다. N-key rollover 한계나 OS/system reserved key를 엔진이 보장한다고 기술하지 않는다.

텍스트를 key code→문자로 직접 변환하지 않는다. Windows text service는 PlatformWindow의 TSF adapter로 분리하고 composition start/update/end, commit, selection, caret rectangle을 계약으로 둔다. 한국어·일본어·중국어 조합, dead key, surrogate/emoji, 조합 취소를 검증한다. OS API의 UTF-16 위치와 엔진 UTF-8 byte 위치를 명시 변환하며 UI cursor는 grapheme 경계를 사용한다. grapheme·BiDi·shaping은 별도 text library adapter 책임이다. D8

macOS text service는 NSTextInputClient를 구현한 native view adapter로 composition/marked text·선택 범위·replacement·commit·caret rectangle을 연결한다. Windows TSF와 같은 TextSessionId/route generation·정확히 한 번 commit·취소 계약을 따르되 vendor API를 통합 enum으로 흉내 내지 않는다. UTF-16 range와 UTF-8 byte/grapheme 위치를 명시 변환한다. 한 route에는 native 또는 웹 text owner 하나만 두고, IME 조합 중 focus/PIE 교체·늦은 commit·dead key·한국어/일본어/중국어·emoji를 두 OS에서 실물 시험한다. D15

19.3 마우스와 pointer

카메라에는 상대 이동 counts, UI에는 논리 좌표를 사용한다. 물리 pixel·logical UI unit·normalized viewport 좌표는 서로 다른 타입 또는 명시 변환 함수로 표현한다. DPI·window rect·viewport rect가 바뀌면 변환 epoch를 갱신한다. multi-monitor·negative desktop coordinate·창 바깥 drag·추가 버튼·고해상도 wheel을 fixture에 포함한다.

CapturePointer/ReleasePointer는 PlatformWindow token으로 제공한다. focus loss·창 파괴·사용자 전환 시 token을 회수한다. 상대 모드 전환 때 발생한 synthetic warp event는 표식과 adapter 정책으로 제외하고 첫 실제 delta까지 폐기하지 않는다. mouse sensitivity의 시간 단위는 counts당 각도이며, 동일 frame delta에 frame delta-time을 다시 곱하지 않는다. pad stick의 각속도·초 단위 적분과 구분한다.

터치·펜·tablet은 I2+ 확장 후보다. pointer ID·pressure·tilt capability를 담을 자리는 확보하되 초기 mouse 구현에 가짜 값을 생성하지 않는다. gesture recognition은 물리 backend와 분리한 상위 processor로 추가한다.

20 게임패드·콘솔·출력 장치

20.1 표준 control와 device profile

face button은 South/East/West/North의 위치 의미로 정의한다. 화면 표시 A/B/Cross/Circle와 confirm/cancel 정책은 device origin·platform·사용자 profile을 통해 결정한다. 같은 문자 A가 모든 장치에서 같은 위치라는 가정을 하지 않는다. trigger는 [0,1], stick은 [-1,1]의 정규화 범위를 사용하고 보정 전 raw 값은 probe 진단에 보존한다.

deadzone·anti-deadzone·response curve·axis invert·sensitivity는 compiled profile의 순서로 한 번 적용한다. radial/axial deadzone을 중복 적용하는 backend+action 조합은 금지한다. stick drift 보정과 trigger threshold/hysteresis는 별도 값이다. unknown model은 검증된 mapping database 또는 사용자 calibration으로 처리한다. SDL의 GUID mapping 기능은 선택 adapter의 참고 구현 경계이며 SDL 타입을 public API에 노출하지 않는다. D9

USB/Bluetooth 연결은 동일 제품이라도 capability가 다를 수 있다. battery·gyro·accelerometer·touchpad·trigger effect·LED·audio route는 개별 capability로 조회한다. 일반 rumble 구현만으로 해당 패드의 모든 기능을 지원한다고 표시하지 않는다.

20.2 사용자와 재연결

DeviceHandle, LocalUserId, PlatformUserId, LocalPlayerId를 분리한다. pad slot 번호를 player 번호나 계정 identity로 저장하지 않는다. pairing은 Host의 사용자 정책이며 device hotplug가 기존 player ownership을 임의 변경하지 않는다. 공동 조작·접근성 장치 합성·다중 seat는 선택 기능으로 추가한다.

disconnect 시 active action은 Cancelled, output은 DeviceGone으로 완료한다. focus loss·시스템 overlay·suspend도 route 정책에 따라 gameplay를 cancel/neutral 처리한다. reconnect/resume에서는 descriptor와 capability를 다시 읽고 generation/epoch를 갱신한다. 이미 눌린 버튼은 release 확인 뒤 다시 활성화하여 메뉴를 닫자마자 발사되는 동작을 방지한다. 장치별 연결 안내는 UI에서 표시하며 InputCore가 modal을 직접 생성하지 않는다.

20.3 플랫폼별 adapter 범위

플랫폼	계획 경계	최초 지원 상태·필요 근거
Windows	GameInput 또는 선택 RawInput; text/window 별도	I1에서 키보드·마우스·검증한 패드 모델만 선언
macOS	AppKit/GameController·NSTextInputClient; text/window 별도	I1 키보드/마우스/검증 pad, I2 IME·route 필수
Linux 등	SDL 또는 native adapter 선택	build target·장비 검증 후 추가, Core 수정 불필요
Steam 환경	raw provider와 semantic action provider 구분	native integration은 I2+ 선택
Xbox 계열	GDK input·user·lifecycle 연결	SDK·개발 장비·해당 프로그램 접근 후 검증
PlayStation 계열	승인된 SDK의 장치·사용자·text·output adapter	파트너 자료 확보 전 계약/mock만 정의
Nintendo 계열	승인된 SDK의 controller topology·lifecycle adapter	플랫폼 접근·장비 확보 후 기능별 검증

표 18

Steam Input native API는 action set/layer와 action origin 개념을 제공한다. 이미 의미 action으로 전달된 입력을 일반 raw binding evaluator에 다시 적용하지 않는다. semantic provider는 ActionId 매핑과 동일한 action phase 계약을 만족하는 별도 진입점이며, raw/semantic 동시 활성화 정책은 profile에 명시한다. D10

SourceKey는 RawControlSource와 SemanticActionSource를 구분한다. semantic action origin을 정확한 물리 event identity로 취급하지 않는다. 기본 정책은 LocalUser와 명시 control domain/action group별 provider owner 하나이며, 혼합 사용은 disjoint ownership이 검증된 profile에 한정한다. semantic phase 정규화 후 raw processor/trigger를 다시 적용하지 않는다. IT18은 물리 origin이 없는 semantic fake와 raw fake의 중복 action을 검증한다.

PlayStation·Nintendo 공개 포털은 개발 접근 절차만 확인하는 근거다. 비공개 SDK 함수명·인증 항목·기기 수 제한을 추정하여 고정하지 않는다. Console adapter conformance에는 user pairing, system UI 전환, suspend/resume, software keyboard, hotplug, glyph policy, output 정지를 포함하되 실제 세부 기준은 승인된 문서로 확정한다. PC에서 콘솔 패드가 동작하는 결과는 콘솔 플랫폼 검증을 대체하지 않는다. D12, D13

20.4 haptics와 센서

```
struct HapticRequest {
    DeviceHandle device;
    OutputOwnerToken owner;
    HapticEffectId effect;
    float intensity;
    Duration ttl;
    uint8 priority;
};

Result<OutputTicket> SubmitHaptics(const HapticRequest&);
Result<void> CancelOutput(OutputOwnerToken);
```

output ticket은 Accepted와 backend 완료 상태를 구분한다. effect는 유한 TTL·사용자 강도·platform cap을 적용하고 channel별 mixer가 합성·clamp한다. stop 명령은 일반 effect queue의 포화로 영구 차단되지 않도록 per-device stop generation mailbox를 사용한다. disconnect는 backend ACK를 무한 대기하지 않고 연결 세대 종료로 해당 ticket을 terminal 처리하되 callback context lease

는 별도 drain한다.

진동·adaptive trigger·LED 등은 독립 feature이며 Unsupported가 정상 결과가 될 수 있다. owner 파괴·focus 정책·pause·shutdown 때 지정 channel을 정지한다. gyro/touch 센서는 좌표계·단위·샘플 시각·calibration version을 명시하고 pose fusion은 선택 processor로 둔다. simulation 재실행으로 haptics를 중복 출력하지 않는다.

21 액션·컨텍스트·재바인딩

21.1 data schema와 평가 API

```
struct ActionDesc {
    ActionId id;
    ActionValueType valueType;
    ActionUpdateDomain domain;
    ActionFlags flags;
};

struct BindingDesc {
    BindingId id;
    ActionId action;
    ControlPath source;
    Span<const ProcessorDesc> processors;
    TriggerDesc trigger;
};

Result<CompiledInputProfile> CompileInputProfile(const InputProfile&, const ProcessorRegistry&,
                                                 Allocator&);
Result<ContextToken> PushInputContext(LocalUserId, const ContextDesc&);
Result<void> ApplyInputProfile(LocalUserId, CompiledInputProfile&&);
Result<RebindToken> BeginRebind(const RebindRequest&);
```

ActionId/BindingId는 stable Guid이며 display name과 분리한다. Boolean/Axis1D/Axis2D/Axis3D를 우선 제공하고 quaternion 등은 capability가 필요할 때 추가한다. ControlPath는 device class·표준 control·선택 instance 조건으로 구성한다. 런타임 매 frame에 문자열 경로를 해석하지 않고 profile compile 단계에서 검증·해석한다.

processor는 input/output value type, parameter schema, version, deterministic flag, compiled state size/alignment를 등록한다. registry 변경 시 기존 compiled profile은 module lease로 함수를 유지하거나 safe boundary에서 재컴파일한다. processor state는 평가 lane 소유이며 backend callback이 수정하지 않는다. 임의 함수 호출 그래프나 Blueprint node 체계를 도입하지 않는다.

21.2 trigger와 context 의미

항목	초기 계약	오류·경계 처리
Press/Release	유효한 down/up transition	focus/장치 손실은 Release trigger가 아니라 Cancelled
Hold	설정 duration 이상 유지	종료 시 Completed, 외부 취소는 Cancelled
Tap/MultiTap	최대 유지·간격을 tick 단위로 평가	확정 시점 명시, hold와 중복 발사 금지
Chord	modifier/control 집합 조건	순서·동시성 허용 창·소비 정책 명시

항목	초기 계약	오류·경계 처리
Axis/Composite	digital WASD·stick 등 합성	합산/최대 절댓값/최근 source 중 정책 선택
Context	priority와 stable tie-break	바뀐 context의 active action은 cancel, source 소유권 재계산

표 19

phase는 Started/Ongoing/Triggered/Completed/Cancelled로 통일하고 action type별 발생 규칙을 golden fixture로 고정한다. Hold의 매 tick Triggered 여부, Tap 판정 지연, chord 해제 시 재시작 여부는 trigger descriptor가 명시한다. trigger 상태 머신의 시간 기준은 GameplayTick 또는 UIRealTime 중 하나이며 time dilation 영향을 숨기지 않는다.

consumption은 raw의 physical source별, semantic의 선언된 SourceKey/owner group별로 결정한다. 높은 context가 key를 소비하면 낮은 context에 동일 down edge를 전달하지 않는다. Shared 플래그는 명시한 action에만 허용한다. priority가 같은 경우 등록 순서 대신 stable context ID를 사용한다. action 간 조합은 DAG로 검증하여 cycle을 거부하고 평가 순서를 고정한다.

21.3 재바인딩과 profile 적용

RebindSession은 대상 user/action, 허용 device class, axis threshold, modifier/chord 정책, timeout, cancel control을 가진다. 시작 시 기존 held control이 neutral이 될 때까지 기다리고 mouse jitter를 후보로 선택하지 않는다. 후보 충돌은 현재 context 및 공유 정책에 따라 report하며 자동 덮어쓰기 여부를 명시한다. 장치 제거·focus 상실·timeout은 세션 취소다.

변경 profile은 validation/compile 성공 후 tick 경계에서 원자 교체한다. 실패하면 기존 profile을 유지한다. 교체 시 active action cancel과 require-release를 적용하여 held key가 새 action을 즉시 발사하지 않게 한다.
default→platform→device→user→accessibility override의 우선순위를 versioned schema로 저장한다. mapping data migration은 ActionId/BindingId를 기준으로 수행하며 이름 변경에 의존하지 않는다.

접근성에는 모든 gameplay action의 재매핑, hold/toggle 대안, 축 반전·감도·진동 강도, 디지털 메뉴 이동, 변경된 binding의 표시를 포함한다. glyph/도움말은 현재 effective binding과 action origin에서 생성한다. Xbox 접근성 가이드의 재매핑·대체 조작 권고를 참고하되 본 계획을 인증 완료로 표현하지 않는다. D11

22 UI·에디터·컷씬·커스텀 언어 연결

22.1 focus와 입력 전달 순서

Platform system reserved 동작 → 활성 window/사용자 → text/IME와 modal UI → 일반 UI → native viewport → gameplay context 순으로 전달 자격을 판정한다. 실제 소비 우선순위는 route policy로 설정하되 하나의 event를 web UI와 native gameplay 양쪽에서 독립 소비하지 않게 한다. UI와 gameplay 모두 필요한 action은 Shared로 명시한다.

웹 기반 에디터의 DOM text focus는 webview가 소유한다. 같은 창의 native TSF가 동일 commit을 다시 전달하지 않는다. native viewport focus token과 webview focus token은 Host가 관리하며, focus 요청에는 WindowEpoch·PIE session·LocalPlayer를 포함한다. viewport 첫 클릭은 기본적으로 focus 획득만 수행하고 gameplay 전달 여부는 명시 옵션이다.

modal을 열면 기존 gameplay action을 취소한다. 닫은 후에는 require-release 규칙을 적용한다. split-screen의 UI focus는 LocalUser별로 관리하며 OS의 foreground window 상태와 분리한다. gamepad navigation은 focusable tree·direction policy·repeat 설정을 사용하고 pointer 좌표가 없다는 이유로 임의 mouse 위치를 생성하지 않는다.

22.2 편집기와 실시간 검사

InputProfileEditor는 Action/Binding/Context 목록, processor parameter property panel, conflict report, 실제 장치 입력 표시, 현재 effective binding/glyph 미리보기를 제공하는 후속 도구다. runtime과 같은 schema·validator·CompileInputProfile을 호출한다. undo/redo는 profile transaction 단위이며 preview 실패가 저장된 profile을 훼손하지 않는다. 첫 I0/I1은 CLI probe와 fixture 파일로 충분하다.

UMG 유사 인게임 UI의 입력 연결은 WidgetFocus/Navigation/TextSession 계약으로 정의한다. designer preview는 synthetic input과 real input을 선택하고 preview route epoch로 실제 게임 세션과 격리한다. UI layout/render/designer 자체는 전체 계획의 UI 모듈 후속 작업이며 InputCore 선행 조건이 아니다.

22.3 컷씬·개발자 콘솔·언어

컷씬은 player control을 직접 bool로 덮어쓰지 않고 InputSuppressionToken/ContextToken을 소유한다. stop·종료형 skip·abort·owner destroy에서 token을 반환한다. 재생을 유지하는 seek에서는 token을 유지하고 active section의 token 집합을 같은 route 경계에서 원자 재조정한다. 이전 token 해제와 새 token 획득 사이에 gameplay 허용 구간을 만들지 않는다. 끝으로 seek하여 playback이 실제 종료될 때만 종료 경로를 적용한다. 중첩 컷씬은 token 집합으로 합성하고 마지막 owner 종료 시 허용 상태를 복원한다. control channel은 AlwaysRestore이며 property의 Keep State 정책과 분리한다. skip action은 허용된 별도 context로 남긴다. IT20은 중간 seek·pause seek·section 전환·종료 seek를 구분한다.

개발자 명령 콘솔은 text consumer와 command dispatcher다. 게임 console hardware backend와 별개다. shipping build에서는 명시적으로 허용한 명령만 노출하며 text focus 중 이동·발사 action이 발생하지 않아야 한다.

커스텀 언어에는 ActionId 조회, action event 구독, context token, rebinding/profile API를 typed binding으로 제공한다. Blueprint와 시각적 gameplay script graph는 지원하지 않는다. subscription은 WeakObjectRef owner와 module/script lease를 보유하고 owner destroy·VM reload·World 종료 때 해제한다. callback은 Input lane에서 바로 VM에 진입하지 않고 지정된 gameplay phase에 event batch를 전달한다. root로 subscription owner를 영구 보존하는 cycle을 만들지 않는다.

23 재생·네트워크·헤드리스 검증

23.1 두 수준의 기록

기록	목적·데이터	제한
Normalized device replay	event 순서·clock quality·device descriptor·InputGap·route 변화	진단 전용; SDK pointer·OS 장치 handle 저장 금지
Semantic command replay	tick·player·ActionId 또는 command ID·quantized value·sequence·profile revision	gameplay 재현; raw text·haptics 제외

표 20

binary header에 schema/processor version, tick rate, coordinate/unit policy, action manifest hash를 기록한다. decoder는 length·count·allocation budget을 검사한다. unknown required action/type은 명시 실패하고 optional channel skip은 선언된 경우에만 허용한다. replay 파일을 읽었다는 이유로 plugin 코드를 자동 로드하지 않는다.

record/replay의 동일성을 인정하는 범위를 구분한다. InputCore의 quantized command 결과는 동일 조건에서 bit-exact를 목표로 한다. 전체 물리·애니메이션·네트워크 simulation의 cross-platform 결정성까지 입력 시스템 단독으로 보장하지 않는다. floating processor는 math policy를 고정하거나 command 직전에 canonical quantization을 적용한다.

23.2 네트워크와 side effect

client는 장치 모델 대신 semantic command를 서버로 전송한다. 서버는 tick window·sequence·범위·rate·플레이 상태를 검증한다. client profile 변경이 서버 권한이나 cooldown을 변경하지 않는다. prediction/resimulation은 이미 기록한 command를 재사용하며 OS 입력을 다시 읽지 않는다.

simulation 상태 변경 함수는 rollback tick마다 재실행한다. haptics·UI sound·외부 IO는 별도 side-effect ledger를 통해 commit하며 stable event identity와 world/session epoch로 중복을 억제한다. 예측 presentation과 confirmed presentation 정책을 구분하고 rollback으로 폐기된 haptic owner는 cancel/TTL로 정리한다. VM bridge는 execution mode를 전달하여 simulation callback이 임의 IO를 중복 발행하지 않게 한다. IT22에서 최종 simulation counter는 oracle와 같고 haptic commit은 1회인지 검사한다. 이 네트워크 연결은 후속 Net 모듈 작업이며 I0의 통과 조건은 command codec과 fake sink다.

23.3 실행 target

FoundationTests와 InputCoreTests는 window/GPU/sound/device driver 없이 실행한다. InputDeviceProbe는 real backend의 enumerate·pressed transition·latency·capability·hotplug를 확인한다. headless test가 성공하였다고 실제 IME·콘솔 lifecycle·8 kHz 장치가 검증되었다고 보고하지 않는다.

전용 서버에서는 물리 backend·haptics·text service를 제거한다. offscreen GPU 렌더 target은 CPU headless와 다른 구성으로 유지한다. 실제 콘솔 입력 검증은 해당 장비 suite이며 synthetic console mock은 공통 계약 검사로만 표시한다.

24 입력 작업 순서와 테스트 기준

24.1 구현 작업

단계	작업	완료 결과
I0-A	I00–I04	schema·Synthetic·queue·clock·device lifecycle·tick snapshot
I0-B	I05–I08	action·trigger/context·profile/rebind·replay
I1	I09–I12	Windows/macOS backend·mouse/focus·pad·각 OS DeviceProbe
I2	I13–I16	TSF/NSTextInputClient IME·UI 연결·haptics·언어/World bridge·profile editor
선택 확장	I17–I19	SDL·Steam semantic·콘솔 adapter 및 실장비 suite

표 21

I00의 계약 검토와 schema 초안은 R1 중 병행 가능하다. InputCore의 실제 구현 착수 기본 시점은 R2 통과 이후다. I00/I01을 먼저 prototype하더라도 Core Gate 완료로 대체하지 않는다. I17–I19의 완료는 I0/I1을 막지 않으며 콘솔 SDK 미확보는 해당 adapter의 명시 미지원으로 유지한다.

24.2 검증 matrix

ID	시나리오	통과 기준
IT01	Synthetic backend 두 종류 등록·해제·재등록	InputCore 수정 없이 등록, late callback 안전, lease 0

ID	시나리오	통과 기준
IT02	한 frame 안 down/up/down; timestamp 동일·역순 수신	결정된 순서와 edge 수 보존, 최종 상태 일치
IT03	30/60/144 FPS·catch-up·동일 user/tick의 UI/PIE A/B scope	동일 command, delta 중복 0, scope/clock 혼합 0 또는 명시 expired
IT04	event queue/payload 강제 포화	InputGap 기록, Cancelled/neutral/resync, stuck key 0
IT05	장치 slot 재사용·USB 제거·재연결	stale generation 거부, require-release, 사용자 오배정 0
IT06	Win Alt-Tab/Mac 앱 전환·focus·modal·suspend/resume	gameplay 취소, 복귀 순간 발사/이동 edge 0
IT07	US/KR 배열·좌우 modifier·repeat·layout 변경	physical/logical/text 분리, repeat 중복 action 0
IT08	Win TSF/Mac NSTextInputClient·한국어/일본어/중국어·emoji·dead key	중복 commit·UTF 위치 오류 0, composition 취소 정상
IT09	8 kHz burst·Win DPI/Mac backing scale·다중 모니터·capture 해제	delta/unit 일치, warp 제외, loss 표시
IT10	pad stick drift·radial deadzone·trigger hysteresis	보정 1회, 임계 경계 golden curve 일치
IT11	Xbox/PS/Nintendo glyph origin fixture	위치·표시·confirm 정책 분리, fallback 표시 명확
IT12	pad 4개·local user 2명·hotplug·계정 변경 mock	ownership 유지, 잘못된 사용자 action 0
IT13	tap/hold/multitap/chord 경계 tick	phase·발생 횟수 golden FSM과 일치
IT14	context priority/tie·source consume·Shared	중복 발사 없음, 결정적 tie-break, cycle 거부
IT15	rebind 시작 held key·axis noise·conflict·장치 제거	neutral 대기, 적절한 후보/취소, 기존 profile 보존
IT16	profile 교체 중 held key·schema migration 실패	tick 경계 원자 적용, require-release, rollback
IT17	haptic queue full·owner destroy·disconnect·stop	stop 누락 0, finite TTL, ticket terminal 1회
IT18	raw/semantic 두 provider의 같은 source	선언한 owner만 전달, Steam action 이중 평가 0
IT19	web/native·PIE A pause/B 진행·재시작·hotplug·지연 query	중복/타 scope 반환 0, 구 epoch 무효, terminal cancel 소비·폐기 추적
IT20	컷씬 중첩·skip·abort·seek·owner destroy	재생 유지 seek는 token 유지, 종료 시 반환·skip context 정합
IT21	record→replay·seek·잘못된 길이/버전	command hash 일치 또는 명시 decode 오류
IT22	server CPU-only·prediction resim·fake output	SDK/GPU 링크 없음, haptic/UI side effect 중복 0
IT23	새 backend·새 processor·새 capability 추가	Core 파일 변경 0, 기존 profile binary fixture 유지
IT24	실제 플랫폼/장치 matrix	SDK·OS·model·transport·capability별 PASS/FAIL/NOT RUN

표 22

I0 필수는 IT01–06·13–16·21–23의 synthetic 가능한 부분이다. I1은 Windows와 macOS 각각 IT07·09–12·24의 선언된 실물 조합을 필수 실행한다. 한 OS에서만 통과하면 I1 전체는 미완료다. IT08·17–20의 실제 통합은 I2/선택 adapter 구현 때 활성화한다. fixture의 mock 통과와 hardware 통과는 결과 필드를 분리한다.

24.3 측정 항목

callback 수신→Input lane drain→action publish→simulation 소비의 p50/p95/p99와 queue high-water·loss·resync 수를 기록한다. 장치 실제 발생 시각이 없는 경우 수신 시점 이후 지연으로 이름을 제한한다. rendering input-to-photon은 별도 camera/display 계측이며 위 CPU timestamp 차이로 대체하지 않는다.

초기 성능 예산은 4 local user·16 device·1,000 compiled binding에서 action 평가 0.2 ms p95, InputCore heap의 frame 중 allocation 0회를 탐색 목표로 둔다. 기준 CPU·build·tick·profile을 고정한 뒤 실측하여 확정한다. 목표 미달이면 profiler 기반으로 최적화하며 입력 손실이나 critical section 생략으로 개선하지 않는다.

I13은 standalone composition/route-token 시험과 실제 Studio/PIE 시험을 분리한다. standalone 성공 시 실제 IT19는 NOT RUN으로 남길 수 있으며 I13 전체 완료를 뜻하지 않는다. 실제 subset에는 양OS WebView2/WKWebView·native viewport·UI/text focus·PIE fixture가 필요하다. I15/I16은 이 외부 조건에 World/VM/cutscene 조건을 누적한다. Foundation/I0의 시작을 상위 Studio 완료에 의존시키지 않는다.

24.4 입력 작업별 선행 관계

ID	Gate·선행 작업	작업·산출물	검증
I00	I0-A / C26	입력 계약·schema — Device/Control/Event/Backend·capability·오류	IT01;IT23
I01	I0-A / I00	Synthetic backend — clock 주입·event fixture·lifecycle mock	IT01;IT02
I02	I0-A / I01	event queue·clock — POD payload lease·정렬·overflow/resync	IT02;IT04
I03	I0-A / I02	device lifecycle — generation·identity·pairing 계약·focus epoch	IT05;IT06;IT12
I04	I0-A / I03	tick snapshot — 한 event 한 tick·catch-up·late event·immutable view	IT03
I05	I0-B / I04	액션 평가 — typed processors·value·compiled binding	IT13;IT23
I06	I0-B / I05	trigger·context — FSM·priority·consumption·cancel/require-release	IT06;IT13;IT14
I07	I0-B / I06	profile·재바인딩 — migration·conflict·neutral·원자 교체	IT15;IT16
I08	I0-B / I07	replay·command codec — quantization·schema hash·headless fake sink	IT21;IT22
I09	I1 / I08	Windows·macOS 장치 adapter — GameInput/RawInput·AppKit/GameController·provider arbitration	IT07;IT18;IT24
I10	I1 / I09	Win/Mac window·mouse 연결 — DPI/backing scale·relative/capture·focus·wheel·main-thread 규약	IT06;IT09
I11	I1 / I09	Win/Mac 기본 gamepad — calibration·glyph origin·hotplug·user pairing·플랫폼 capability	IT05;IT10;IT11;IT12
I12	I1 / I10;I11	Win/Mac DeviceProbe·실물 matrix — 두 필수 OS의 capability/latency/loss·지원 패드·장치 검증	IT24
I13	I2 / I12	Win/Mac text·IME·UI route — TSF/NSTextInputClient·composition·selection/caret·web/native focus	IT08;IT19(standalone/Studio subset 구분)
I14	I2 / I12	haptics·output — finite TTL·mixer·stop mailbox·ticket terminal	IT17
I15	I2 / I13;I14	게임/VM/컷씬 bridge — weak subscription·context token·side-effect ledger	IT19;IT20;IT22

ID	Gate·선행 작업	작업·산출물	검증
I16	I2 / I07;I15	InputProfileEditor — schema 기반 편집·undo·검증·preview	IT15;IT16;IT19
I17	선택 I2+ / I12	SDL adapter — 선택 플랫폼 mapping·conformance·장치 matrix	IT01;IT23;IT24
I18	선택 I2+ / I12	Steam semantic adapter — action ID/origin·raw/semantic arbitration	IT18;IT24
I19	선택 I2+ / I13;I14	콘솔 adapter — SDK별 lifecycle·user·input/output·실장비 검증	IT05;IT06;IT11;IT12;IT17;IT24

표 23

25 확장 검증·위험·보류 목록

25.1 새 기능 추가 절차

공통 의미와 capability·version·ownership·실패 상태를 ADR로 정의한다.
해당 module의 descriptor·factory·schema를 추가한다. Core 변경 필요 여부를 검토한다.
synthetic/reference fixture와 supported/unsupported 결과를 먼저 구현한다.
production adapter를 연결하고 lifecycle·thread·reload·failure conformance를 실행한다.
기존 binary/profile fixture와 disabled target 조합을 회귀 검사한다.
지원 matrix에 검증한 조합만 추가하고 optional 기능의 미지원 사유를 남긴다.
Extensibility Gate는 새 allocator, 새 reflected type, 새 field codec, 새 input backend, 새 action processor를 별도 test module에서 추가하는 다섯 시나리오다. 기존 Core source 수정 없이 가능해야 하며 public header에 SDK 타입이 노출되지 않아야 한다. 신규 schema 의미를 위한 공통 변경이 필요하면 예외 ADR과 migration fixture를 동반한다. dynamic unload는 이를 검증하는 유일한 방법이 아니며 static registration으로 초기 Gate를 완료할 수 있다.

25.2 위험과 대응 우선순위

우선	위험	초기 대응·완화 확인
P0	GC 누락 edge·root race·부분 생성 노출	full oracle, gate 선형화, failure injection
P0	STL 대체 범위 확대로 첫 객체 구현 지연	최소 container·수동 schema·full STW부터 완료
P0	Core/Platform·Object/World 순환	Base 분리, link/include audit, opaque context
P0	초기 확장 API의 소유권 불명확	token/lease·drain·Unsupported 규약 필수
P1	모든 backend를 먼저 추상화하는 과설계	Synthetic+실제 1종으로 경계 검증 후 확장
P1	input focus·hotplug·queue 손실의 stuck state	Cancelled/neutral/require-release 공통 state machine
P1	커스텀 언어·메타 생성기 동시 착수 과부하	manual TypeInfo 먼저, VM bridge는 후속
P1	public template/include 확대에 따른 빌드 비용	module 측정·PImpl 선택·변경 영향·cache 적중 검증
P2	console SDK/장비 미확보	명시 Unsupported, mock과 실기 검증 분리

우선	위험	초기 대응·완화 확인
P2	lock-free·fast allocator 조기 적용	mutex 기준 구현 보존, 성능 증거 후 대체

표 24

25.3 반복 작업과 빌드 비용

작업 시작 시 header fan-out, TU별 compile 시간, link 시간, generated 파일 변경 수, cache hit/miss를 저장한다. schema 변경 없는 구현 편집이 전체 engine rebuild를 유발하지 않아야 한다. generated 코드는 내용이 같은 경우 timestamp를 유지하고 public reflection header를 모든 module에 포함하지 않는다.

초기에는 CMake/Ninja와 작은 target으로 반복 경로를 확보한다. 자체 Rocks Build·distributed executor·Incredibuild adapter는 같은 target/action manifest 위에서 후속 연결한다. action key에는 compiler·SDK·명령·환경 allowlist·입력 closure를 포함하고 negative include 변화도 재검사한다. Unity build·PCH·C++ module·LTO는 독립 옵션으로 비교하며 디버그 반복에서는 incremental link/분할 library, release에서는 최적화 link를 별도 구성한다. 중앙 scheduler나 remote cache 없이도 local fallback으로 재현할 수 있어야 한다.

C31은 계약과 선택 최적화 작업이며 R1의 실행 선행 조건이 아니다. cache가 실제 오류를 은폐하지 않는지 clean/cached output 비교·source add/remove·missing include 생성·toolchain 변경 fixture를 사용한다. 빌드 최적화는 기존 전체 계획의 상세 항목과 연결하며 분산 환경이 준비되지 않아도 로컬 개발을 계속할 수 있다.

Windows/macOS 신규 checkout의 scaffold→Core build/test→headless report를 C00/26 공통 smoke로 유지한다. 두 OS에서 path separator·대소문자·Unicode·process exit·dynamic library suffix·toolchain/SDK cache key를 검증한다. Windows 전용 build executor가 없어도 Mac local build가 동작해야 하며 remote executor 지원 여부와 필수 local 개발 기능을 분리한다.

25.4 초기 보류 항목

fully concurrent GC, lock-free registry, moving/compacting heap, runtime native layout patch, full VM, 범용 hot unload, 모든 콘솔 SDK, 고급 센서 fusion, 모든 패드의 특수 output, 전용 InputProfileEditor, 모든 언어의 text shaping을 첫 R1/I0 완료에 요구하지 않는다. 메모리 안전·오류 전파·기본 ownership·버전·확장 경계는 보류하지 않는다.

26 전체 계획과의 연결·검토 기록·참고 자료

26.1 기존 요구의 유지

이 상세안은 Rocks 전체 계획을 대체하는 축소 사양이 아니다. tags·component·prefab·asset/cook/DDC·WorldPartition·네트워크·오디오·AI/nav·physics·animation·VFX·UMG 유사 UI·컷씬·웹 에디터·native viewport는 Object/Schema/Task의 후속 소비자다. Deferred/Forward·upscaling·GPU-driven geometry/GI·high-end post effect는 renderer 계획에 유지한다. Blueprint 제외·커스텀 언어 정책도 유지한다.

후속 시스템	이번에 확보할 연결점	지금 구현하지 않아도 되는 부분
Entity/Component/Tags/Prefab	Type/Field/Subobject ID·owner edge·clone/default	World lifecycle·tag editor·prefab importer
Asset/Async/Cook/DDC	Archive·stable ID·IO lease·Task·schema version	asset database·cook farm·package format 전체
Renderer/Audio/Physics	immutable snapshot·backend completion token	실제 GPU/audio/physics resource manager

후속 시스템	이번에 확보할 연결점	지금 구현하지 않아도 되는 부분
WebEditor/UMG/Cutscene	schema property·transaction·input focus/context	편집 화면·widget render·timeline
Material Editor·Compiler	stable ID·schema·Task·snapshot·module lease	필수 M3/M4 그래프·preview·instance/function; M5/M6 layer
Custom Language	TypeSchema·typed binding·root provider·weak subscription	VM/compiler/debugger
Net/AI/Navigation	quantized command·scheduler·context epoch	replication/prediction·AI runtime·nav query

표 25

머티리얼 에디터는 반드시 구현할 후속 제품 기능이다. 그래프 편집·인스턴스·함수·레이어·native live preview·비동기 compile·DDC/cook를 전체 계획의 14.5–14.9절에 정의한다. Blueprint 제외는 gameplay visual scripting에 적용하며 shader 제작 그래프는 유지한다. C++/커스텀 언어는 parameter API로 재질을 제어한다.

Core 단계에서는 stable Guid·versioned schema·Archive·Task 취소/terminal·immutable snapshot·module lease·transaction 연결 계약을 준비한다. MaterialSchema·MaterialCompiler·MaterialRuntime·EditorMaterial은 별도 상위 모듈이며 Core가 이 모듈을 include/link하지 않는다. M3/M4의 MAT0–MAT2 기본 에디터는 필수이고 layer 고도화는 M5/M6, 미지원 domain/target은 명시 상태로 확장한다. Core/Input 작업 ID 52개는 유지하되 예정 파일 수는 생성기에서 실제 집계한다. 상위 제작 기능 전체를 첫 Foundation 완료 조건에 추가하지 않는다.

26.2 최신 전체 요구와 후속 Gate

전체 요구	Core/Input의 선행 계약	필수 후속·검증 시점
제작 템플릿·사용자 카탈로그	C00 최소 scaffold·stable ID·transaction·무변경 출력 유지	M3 project/module/test·M4 asset/UI/input·M5 사용자 저장/Linked 갱신; TT02–08
PIE Play/Pause/Resume/SingleStep/Stop	context epoch·clone/remap·Core clock·input route·cancel/lease·누수 ledger	M3 기본·M4 정식, 2개 이상 session 격리·1,000회 종료·PI01–06/NL04
variable/fixed tick·time dilation·병렬 tick	C15 ClockService/StepPlanner·C17 DAG/owner commit	M3/M4 World/physics/PIE 연결, TK01–06
필수 로그·프로파일러·상태 조회	C06 최소·R1 ledger·C18 analyzer/viewer·C26 report	Rocks Lens 편집기 통합과 GPU/network/audio/animation별 최초 Gate; OB/NL
material·lightmap bake·probe 조명	schema/Task/IO·math adapter·revision·cancel/publish	material M3/M4, lightmap/probe M4 이후; CPU 기준/실제 renderer 검증
texture streaming·기본 콘텐츠	allocation budget·lease·IO/decode·stable ID	최소 mip residency/손상·취소, built-in lifetime/baseline을 실제 asset/renderer Gate에서 검증
5개 graphics API·Deferred/Forward·upscale·고급 post	immutable snapshot·backend completion·capability·시간 history ID	D3D11/D3D12/Vulkan/OpenGL/Metal 각각 지원표·실장비 conformance; 기능별 일정은 전체 계획

전체 요구	Core/Input의 선행 계약	필수 후속·검증 시점
웹 editor/native viewport·인게임 UI 편집·컷씬	Win/Mac focus/IME·transaction·schema·clock·PIE scope	UMG 계열 designer/runtime·native preview·sequence authoring/capture 필수; 자체 언어 이벤트
Build/Cook/DDC·분산/반복 최적화	C00 manifest·C20 내용 동일 유지·C31 action/cache 계약	Rocks Build·분산 executor·로컬 fallback·compile/link/cook 측정; headless 필수
전체 gameplay/AI/VFX/network/audio/nav/physics	Object/Schema/Task/context·ownership·테스트 하네스	각 subsystem 최초 구현부터 unit/contract/fault/concurrency·관측·누수 Gate

표 26

엔진 내부 명명은 Rocks 고유 체계를 유지하고 Blueprint는 지원하지 않는다. 자체 언어와 material/UI/animation의 전문 편집기를 구분한다. 모든 후속 구현의 검증 의무와 안정성 우선은 Foundation 이후에도 유지한다. 위 표는 구현 완료 선언이나 별도 신규 Core task가 아니며 전체 계획의 마일스톤으로 연결한다.

AssetLease는 후속 loader의 정상 retention과 owner scope revoke를 구분한다. C24에서는 fake owner의 retention/late completion/revoke를 검증하고, 실제 package admission·공유 revision·GPU 작업은 전체안 9.2의 M3 이후 NL03/NL04·PI05 통합 사례로 검증한다.

26.3 교차 검토 반영

Core/Object, 입력 수명·통합, 빌드·작업 순서를 세 검토자가 독립 검토하였다. RootRecord scan 수명, commit 전후 실패 경계, ContextId 전달, callback 재진입 후 논리 상태 검사를 보완하였다. 이는 설계 문서 검토 결과이며 코드·성능 테스트 통과를 의미하지 않는다. 입력 손실 복구 epoch, route/text session 세대, 컷씬 seek, raw/semantic ownership, event batch, resim 효과 분리를 추가하였다. 초기 Function/Guid 도입 시점, Archive bridge, 단계별 테스트 범위도 정정하였다. 작업·파일 목록은 의존 DAG와 경로 중복을 별도로 검사한다.

최종 통합본은 서브에이전트 3개가 4차례 검증하였다. 담당을 교환한 2차 검토에서 task payload·metadata deep copy·input scope·최초 Sync.h·formatter key 등을 보완하고, 3차에서 포화 중 cleanup 진척과 callback 재진입을 추가 보완하였다. 4차는 완료 handle 보관·running cancel·RootRef/ModuleLease 해제·source arena poison·동일 tick의 다중 scope·callback 자기 Stop·초기 파일 의존성과 JSON/CSV 일치를 다시 확인하였다. 전체안과 합친 24개 지적은 문서에 반영되었으며 검토 범위의 미해결 항목은 없다. 실제 구현·실행·장치 검증은 각 작업의 하네스로 수행한다.

문서의 모든 API는 signature 초안이다. 구현 PR은 이 규약을 유지한 구체 header와 실제 compile fixture를 제출해야 한다. 파일 목록의 경로는 생성 예정이며 존재하는 엔진 소스로 오인하지 않는다. CSV의 상태는 계획/선택 후속이며 자동 완료로 간주하지 않는다.

26.4 참고 자료

외부 문서는 언어·도구·플랫폼 API 사실의 근거다. 모듈 경계·API·Gate·수치 목표·일정은 본 엔진의 설계 제안이다. 확인일은 2026.11.10이며 SDK·toolchain 버전은 구현 시 lockfile에 고정한다. 비공개 콘솔 자료는 조회하지 않았다.

ID	공식·원문 자료	적용 범위
D1	C++23 N4950 basic.types	객체 표현·trivially-copyable; 공개 draft mirror
D2	Clang LibTooling	AST 기반 host 메타 도구
D3	Clang JSON Compilation Database	target별 compile command 입력
D4	Epic Incremental Garbage Collection	증분 GC barrier 위험 참고
D5	Microsoft GameInput Overview	PC/Xbox 경로·통합 입력 API
D6	Microsoft GameInput Fundamentals	reading/시간·이벤트 수집 모델

ID	공식·원문 자료	적용 범위
D7	Microsoft About Raw Input	Windows keyboard/mouse adapter
D8	Microsoft Text Services Framework	text/IME 서비스 경계
D9	SDL3 AddGamepadMapping	선택 mapping backend
D10	Steam Input General Concepts	semantic action·set/layer·origin
D11	Xbox Accessibility Guideline 107	재매핑·대체 조작 권고
D12	PlayStation Partners	개발 접근 경로, 비공개 SDK 미확인
D13	Nintendo Developer Registration	개발 접근 경로, 비공개 SDK 미확인
D14	Clang ThreadSanitizer	지원 target에 따른 race 검사
D15	Apple NSTextInputClient	macOS native text·marked/selected range·commit 경계
D16	Apple Game Controller	macOS 장치 adapter의 공식 API 경계
D17	GLM 공식 저장소	검증할 math backend 후보; 선정 미완료
D18	Realtime Math 공식 저장소	검증할 math backend 후보; 선정 미완료
D19	DirectXMath 공식 저장소	검증할 math backend 후보; 품질 배제·선정 확정 없음

표 27

EOD